

FRE7241 Algorithmic Portfolio Management

Lecture#5, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

November 25, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

Optimal Stock Weights

If an investor's utility of wealth is logarithmic:
 $u(w) = \log(w)$, then to maximize their utility, they
 should invest the fraction k_f of their wealth in a stock:

$$k_f = \frac{\bar{r}}{\sigma^2}$$

Where $\bar{r}$ is the expected stock return and σ^2 is the expected variance. The fraction k_f is called the *Kelly fraction* or the *Kelly ratio*.

The expected *logarithmic utility* u can be expanded to second order as:

$$\begin{aligned} u &= \mathbb{E}[\log(1 + k_f r)] = \mathbb{E}\left[\left(k_f r - \frac{(k_f r)^2}{2}\right)\right] = \\ &= k_f \bar{r} - \frac{k_f^2 \sigma^2}{2} \end{aligned}$$

The *Kelly ratio* which maximizes the utility is found by equating the derivative of utility to zero:

$$\frac{du}{dk_f} = \bar{r} - k_f \sigma^2 = 0 \rightarrow k_f = \frac{\bar{r}}{\sigma^2}$$

The *Kelly criterion* favors stocks with low returns. Given two stocks with the same *Sharpe* ratios, the stock with the lower returns will have a higher *Kelly ratio*.

```
> # Objective function equal to the sum of returns
> objfun <- function(retp) sum(na.omit(retp))
> # Objective function equal to the Sharpe ratio
> objfun <- function(retp) {
+   retp <- na.omit(retp)
+   if (NROW(retp) > 12) {
+     stdev <- sd(retp)
+     if (stdev > 0) mean(retp)/stdev else 0
+   } else 0
+ } ## end objfun
> # Objective function equal to the Kelly ratio
> objfun <- function(retp) {
+   retp <- na.omit(retp)
+   if (NROW(retp) > 12) {
+     varv <- var(retp)
+     if (varv > 0) mean(retp)/varv else 0
+   } else 0
+ } ## end objfun
```

In portfolio optimization, assuming zero correlations, the *maximum Sharpe* portfolio weights are proportional to the *Kelly ratios*:

$$w_i \propto \frac{\bar{r}_i}{\sigma_i^2}$$

This is also a consequence of logarithmic utility.

Portfolio Weight Constraints

Constraints on the portfolio weights are applied to satisfy investment objectives and risk limits.

Let w_i be the portfolio weights produced by a model, which may not satisfy the constraints, so they must be transformed into new weights: w'_i .

For example, the weights can be centered so their sum is equal to 0: $\sum_{i=1}^n w'_i = 0$, by shifting them by their mean value:

$$w'_i = w_i - \frac{1}{n} \sum_{i=1}^n w_i$$

The advantage of centering is that it produces portfolios that are more risk neutral - less long or short risk.

The disadvantage is that it shifts the mean of the weights, and it allows highly leveraged portfolios, with very large positive and negative weights.

```
> # VTI returns
> retv <- na.omit(rutils::etfenv$returns$VTI)
> datev <- zoo::index(retv) ## Dates vector
> # Load daily S&P500 percentage stock returns
> load(file="/Users/jerzy/Develop/lecture_slides/data/sp500_returns")
> retp <- retstock[datev]
> nrow <- NROW(retp) ## number of rows
> nstock <- NCOL(retp) ## number of stocks
> # Objective function equal to the Kelly ratio
> objfun <- function(retp) {
+   retp <- na.omit(retp)
+   if (NROW(retp) > 12) {
+     varv <- var(retp)
+     if (varv > 0) mean(retp)/varv else 0
+   } else 0
+ } ## end objfun
> # Calculate performance statistics for all stocks
> perfstat <- sapply(retp, objfun)
> sum(is.na(perfstat))
> sum(!is.finite(perfstat))
> hist(perfstat, breaks=100, main="Performance Statistics")
> sort(perfstat, decreasing=TRUE)
```

Quadratic Weight Constraint

Another way of satisfying the constraints is by scaling (multiplying) the weights by a factor.

Under the *quadratic* constraint, the sum of the *squared* weights is equal to 1: $\sum_{i=1}^n w_i'^2 = 1$, after they are scaled:

$$w_i' = \frac{w_i}{\sqrt{\sum_{i=1}^n w_i^2}}$$

Scaling the weights modifies the portfolio *leverage* (the ratio of the portfolio risk divided by the capital), while maintaining the relative weights.

The disadvantage of the *quadratic* constraint is that it can produce portfolios with very low leverage.

```
> # Calculate weights proportional to performance statistic
> # With quadratic constraint
> weightv <- perfstat/sqrt(sum(perfstat^2))
> sum(weightv^2)
> sum(weightv)
> weightv
```

Linear Weight Constraint

A widely used constraint is setting the sum of the weights equal to 1: $\sum_{i=1}^n w'_i = 1$, by dividing them by their sum:

$$w'_i = \frac{w_i}{\sum_{i=1}^n w_i}$$

The *linear* constraint is equivalent to distributing a unit of capital among a stock portfolio.

The disadvantage of the *linear* constraint is that it has a long risk bias. When the sum of the weights is negative, it switches their sign to positive.

```
> # Calculate weights proportional to performance statistic
> # With linear constraint
> weightv <- perfstat/sum(perfstat)
> sum(weightv^2)
> sum(weightv)
> weightv
```

Volatility Weight Constraint

The weights can be scaled to satisfy a volatility target.

For example, they can be scaled so that the in-sample portfolio volatility σ is the same as the volatility of the equal weight portfolio σ_{ew} :

$$w'_i = \frac{\sigma_{ew}}{\sigma} w_i$$

This produces portfolios with a leverage corresponding to the current market volatility.

Or the weights can be scaled so that the in-sample portfolio volatility σ is equal to a target volatility σ_t :

$$w'_i = \frac{\sigma_t}{\sigma} w_i$$

This produces portfolios with a volatility close to the target, irrespective of the market volatility.

Averaging the stock returns using the function `rowMeans()` with `na.rm=TRUE` is equivalent to rebalancing the portfolio so that stocks with NA returns have zero weight.

The function `HighFreq::mult_mat()` multiplies the rows or columns of a *matrix* times a *vector*, element-wise.

```
> # Calculate the weighted returns using transpose
> retw <- t(t(retp)*perfstat)
> # Or using Rcpp
> retf <- HighFreq::mult_mat(perfstat, retp)
> all.equal(retw, retf, check.attributes=FALSE)
> # Calculate the in-sample portfolio volatility
> volis <- sd(rowMeans(retw, na.rm=TRUE))
> # Calculate the equal weight portfolio volatility
> volew <- sd(rowMeans(retp, na.rm=TRUE))
> # Apply the volatility constraint
> weightv <- volew*perfstat/volis
> # Calculate the in-sample portfolio volatility
> retw <- t(t(retp)*weightv)
> all.equal(sd(rowMeans(retw, na.rm=TRUE)), volew)
> # Apply the volatility target constraint
> volt <- 0.01
> weightv <- volt*perfstat/volis
> retw <- t(t(retp)*weightv)
> all.equal(sd(rowMeans(retw, na.rm=TRUE)), volt)
> # Compare speed of R with Rcpp
> library(microbenchmark)
> summary(microbenchmark(
+   trans=t(t(retp)*perfstat),
+   rcpp=HighFreq::mult_mat(perfstat, retp),
+   times=10))[, c(1, 4, 5)]
```

Box Constraints

Box constraints limit the individual weights, for example: $0 \leq w_i \leq 1$.

Box constraints are often applied when constructing long-only portfolios, or when limiting the exposure to certain stocks.

```
> # Box constraints  
> weightv[weightv > 1] <- 1  
> weightv[weightv < 0] <- 0  
> weightv
```

Momentum Portfolio Weights

The portfolio weights of *momentum* strategies can be calculated based on the past performance of the assets in many different ways:

- Invest equal dollar amounts in the top n best performing stocks and short the n worst performing stocks,
- Invest dollar amounts proportional to the past performance - purchase stocks with positive performance, and short stocks with negative performance,
- Apply the weight constraints.

The *momentum* weights can then be applied in the out-of-sample interval.

```
> # Calculate the performance statistics for all stocks
> perfstat <- sapply(retp, objfun)
> sum(is.na(perfstat))
> # Calculate the best and worst performing stocks
> perfstat <- sort(perfstat, decreasing=TRUE)
> topstocks <- 10
> symbolb <- names(head(perfstat, topstocks))
> symbolw <- names(tail(perfstat, topstocks))
> # Calculate equal weights for the best and worst performing stocks
> weightv <- numeric(NCOL(retp))
> names(weightv) <- colnames(retp)
> weightv[symbolb] <- 1
> weightv[symbolw] <- (-1)
> # Calculate weights proportional to the performance statistic
> weightv <- perfstat
> # Center weights so sum is equal to 0
> weightv <- weightv - mean(weightv)
> # Scale weights so sum of squares is equal to 1
> weightv <- weightv/sqrt(sum(weightv^2))
> # Calculate the in-sample momentum strategy pnls
> pnls <- t(t(retp)*weightv)
> # Or using Rcpp
> pnls <- HighFreq::mult_mat(weightv, retp)
> all.equal(pnls, pnls, check.attributes=FALSE)
> pnls <- rowMeans(pnls, na.rm=TRUE)
> pnls[1] <- 0
> # Scale the pnls so their volatility is the same as equal weight
> retew <- rowMeans(retp, na.rm=TRUE)
> retew[1] <- 0
> pnls <- sd(retew)/sd(pnls)*pnls
> wealthv <- xts(cbind(retew, pnls), datev)
> colnames(wealthv) <- c("EqualWeight", "Momentum")
> dygraph(cumsum(wealthv))
```

Rolling Momentum Strategy

In a *rolling momentum strategy*, the portfolio is rebalanced periodically and held out-of-sample.

Momentum strategies can be *backtested* by specifying the portfolio rebalancing frequency, the formation interval, and the holding period:

- Specify a portfolio of stocks and their returns,
- Calculate the *end points* for portfolio rebalancing,
- Define an objective function for calculating the past performance of the stocks,
- Calculate the past performance over the *look-back* formation intervals,
- Calculate the portfolio weights from the past (in-sample) performance,
- Calculate the out-of-sample momentum strategy returns by applying the portfolio weights to the future returns,
- Apply a volatility scaling factor to the out-of-sample returns,
- Calculate the transaction costs and subtract them from the strategy returns.

```
> # Calculate a vector of monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")
> npts <- NROW(endd)
> # Perform loop over the end points
> lookb <- 8 ## Look-back interval in months
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1
> pnls <- mclapply(3:(npts-1), function(tday) {
+   ## Select the look-back returns
+   startp <- endd[max(1, tday-lookb)]
+   retis <- retp[startp:endd[tday], ]
+   ## Calculate the best and worst performing stocks in-sample
+   perfstat <- sapply(retis, objfun)
+   perfstat <- sort(perfstat, decreasing=TRUE)
+   symbolb <- names(head(perfstat, topstocks))
+   symbolw <- names(tail(perfstat, topstocks))
+   ## Calculate the momentum weights
+   weightv <- numeric(NCOL(retp))
+   names(weightv) <- colnames(retp)
+   weightv[symbolb] <- 1
+   weightv[symbolw] <- (-1)
+   ## Calculate the in-sample momentum Pnls
+   pnls <- HighFreq::mult_mat(weightv, retis)
+   pnls <- rowMeans(pnls, na.rm=TRUE)
+   ## Scale weights so in-sample pnl volatility is same as equal w
+   weightv <- weightv*sd(rowMeans(retis, na.rm=TRUE))/sd(pnls)
+   ## Calculate the out-of-sample momentum returns
+   pnlos <- HighFreq::mult_mat(weightv, retp[(endd[tday]+1):endd[tday]])
+   pnlos <- rowMeans(pnlos, na.rm=TRUE)
+   drop(pnlos)
+ }, mc.cores=ncores) ## end mclapply
> pnls <- rutils::do_call(c, pnls)
```

Performance of Stock Momentum Strategy

The initial stock momentum strategy underperforms the index because of a poor choice of the model parameters.

The momentum strategy may be improved by a better choice of the model parameters: the length of look-back interval and the number of stocks.

```
> # Calculate the average of all stock returns
> retew <- rowMeans(retp, na.rm=TRUE)
> # Add initial startup interval to the momentum returns
> pnls <- c(retew[endd[1]:endd[3]], pnls)
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls)
> wealthv <- xts::xts(wealthv, order.by=datev)
> colnames(wealthv) <- c("EqualWeight", "Strategy")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



```
> # Plot dygraph of stock index and momentum strategy
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Stock Index and Momentum Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Momentum Strategy Functional

Performing a *backtest* allows finding the optimal *momentum* (trading) strategy parameters, such as the *look-back interval*.

The function `btmomtop()` simulates (backtests) a *momentum strategy* which buys equal dollar amounts of the best performing stocks.

The function `btmomtop()` can be used to find the best choice of *momentum strategy* parameters.

```
> btmomtop <- function(retp, objfun, lookb=12, rebalf="months", top
+   bidask=0.0, endd=rutils::calc_endpoints(retp, interval=rebalf),
+   ## Perform loop over end points
+   npts <- NROW(endd)
+   pnls <- lapply(3:(npts-1), function(tday) {
+     ## Select the look-back returns
+     startp <- endd[max(1, tday-lookb)]
+     retis <- retp[startp:endd[tday], ]
+     ## Calculate the best and worst performing stocks in-sample
+     perfstat <- sapply(retis, objfun)
+     perfstat <- sort(perfstat, decreasing=TRUE)
+     symbolb <- names(head(perfstat, topstocks))
+     symbolw <- names(tail(perfstat, topstocks))
+     ## Calculate the momentum weights
+     weightv <- numeric(NCOL(retp))
+     names(weightv) <- colnames(retp)
+     weightv[symbolb] <- 1
+     weightv[symbolw] <- (-1)
+     ## Calculate the in-sample momentum pnls
+     pnls <- HighFreq::mult_mat(weightv, retis)
+     pnls <- rowMeans(pnls, na.rm=TRUE)
+     ## Scale weights so in-sample pnl volatility is same as equal
+     weightv <- weightv*sd(rowMeans(retis, na.rm=TRUE))/sd(pnls)
+     ## Calculate the out-of-sample momentum returns
+     pnlos <- HighFreq::mult_mat(weightv, retp[(endd[tday]+1):endd
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     drop(pnlos)
+   }) ## end lapply
+   pnls <- rutils::do_call(c, pnls)
+   pnls
+ } ## end btmomtop
```

Optimization of Momentum Strategy Parameters

The performance of the *momentum* strategy depends on the length of the *look-back interval* used for calculating the past performance.

Research indicates that the optimal length of the *look-back interval* for momentum is about 8 to 12 months.

The dependence on the length of the *look-back interval* is an example of the *bias-variance tradeoff*.

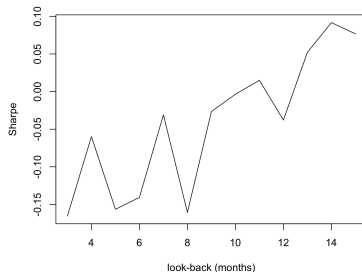
If the *look-back interval* is too long, then the data has large *bias* because the distant past may have little relevance to today.

But if the *look-back interval* is too short, then there's not enough data, and estimates will have high *variance*.

Performing many *backtests* on multiple trading strategies identifying inherently unprofitable trading strategies as profitable, purely by chance (known as *p-value hacking*).

```
> # Perform backtests for vector of look-back intervals
> lookbv <- seq(3, 15, by=1)
> endd <- rutils::calc_endpoints(retp, interval="months")
> # Warning - takes very long
> # pnll <- lapply(lookbv, btmomtop, retp=retp, endd=endd, objfun=objfun)
> # Perform parallel loop under Mac-OSX or Linux
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1
> pnll <- mclapply(lookbv, btmomtop, retp=retp, endd=endd, objfun=objfun, mc.cores=ncores)
> sharper <- sqrt(252)*sapply(pnll, function(pnl) mean(pnl)/sd(pnl))
```

Momentum Sharpe as Function of Look-back Interval



```
> # Plot Sharpe ratios of momentum strategies
> plot(x=lookbv, y=sharper, t="l",
+      main="Momentum Sharpe as Function of Look-back Interval",
+      xlab="look-back (months)", ylab="Sharpe")
```

Optimal Stock Momentum Strategy

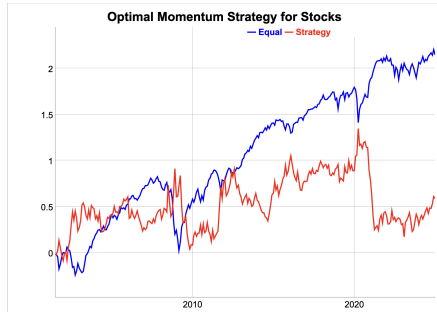
The best stock momentum strategy underperforms the index because of a poor choice of the model type.

But using a different rebalancing frequency in the *backtest* can produce different values for the optimal trading strategy parameters.

The *backtesting* redefines the problem of finding (tuning) the optimal trading strategy parameters, into the problem of finding the optimal *backtest* (meta-model) parameters.

But the advantage of using the *backtest* meta-model is that it can reduce the number of parameters that need to be optimized.

```
> # Calculate best pnls of momentum strategy
> whichmax <- which.max(sharper)
> lookbv[whichmax]
> pnls <- pnll[[whichmax]]
> # Add initial startup interval to the momentum returns
> pnls <- c(retew[enndd[1]:enndd[3]], pnls)
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls)
> wealthv <- xts::xts(wealthv, order.by=datev)
> colnames(wealthv) <- c("EqualWeight", "Strategy")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



```
> # Plot dygraph of stock index and momentum strategy
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Weighted Momentum Strategy Functional

Performing a *backtest* allows finding the optimal *momentum* (trading) strategy parameters, such as the *look-back interval*.

The function `btmomweight()` simulates (backtests) a *momentum strategy* which buys dollar amounts proportional to the past performance of the stocks.

The function `btmomweight()` can be used to find the best choice of *momentum strategy* parameters.

```
> btmomweight <- function(retp, objfun, lookb=12, rebalf="months",
+   bidask=0.0, endd=rutils::calc_endpoints(retp, interval=rebalf),
+   ## Perform loop over end points
+   npts <- NROW(endd)
+   pnls <- lapply(3:(npts-1), function(tday) {
+     ## Select the look-back returns
+     startp <- endd[max(1, tday-lookb)]
+     retis <- retp[startp:endd[tday], ]
+     ## Calculate weights proportional to performance
+     perfstat <- sapply(retis, objfun)
+     weightv <- perfstat
+     ## Calculate the in-sample portfolio returns
+     pnlis <- HighFreq::mult_mat(weightv, retis)
+     pnlis <- rowMeans(pnlis, na.rm=TRUE)
+     ## Scale weights so in-sample pnl volatility is same as equal
+     weightv <- weightv*sd(rowMeans(retis, na.rm=TRUE))/sd(pnlis)
+     ## Calculate the out-of-sample momentum returns
+     pnlos <- HighFreq::mult_mat(weightv, retp[(endd[tday]+1):endd[
+     pnlos <- rowMeans(pnlos, na.rm=TRUE)
+     drop(pnlos)
+   }) ## end lapply
+   rutils::do_call(c, pnls)
+ } ## end btmomweight
```

Optimal Weighted Stock Momentum Strategy

The stock momentum strategy produces a similar absolute return as the index, and also a similar Sharpe ratio.

The advantage of the momentum strategy is that it has a low correlation to stocks, so it can provide significant risk diversification when combined with stocks.

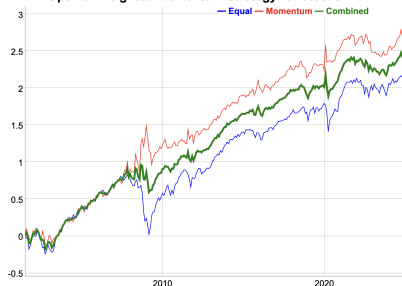
But using a different rebalancing frequency in the *backtest* can produce different values for the optimal trading strategy parameters.

The *backtesting* redefines the problem of finding (tuning) the optimal trading strategy parameters, into the problem of finding the optimal *backtest* (meta-model) parameters.

But the advantage of using the *backtest* meta-model is that it can reduce the number of parameters that need to be optimized.

```
> # Perform backtests for vector of look-back intervals
> lookbv <- seq(3, 15, by=1)
> # pnll <- lapply(lookbv, btmomweight, retp=retp, endd=endd, objf)
> # Or perform parallel loop under Mac-OSX or Linux
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1
> pnll <- mclapply(lookbv, btmomweight, retp=retp, endd=endd, objf)
> sharper <- sqrt(252)*sapply(pnll, function(pnl) mean(pnl)/sd(pnl))
> # Plot Sharpe ratios of momentum strategies
> plot(x=lookbv, y=sharper, t="l",
+      main="Momentum Sharpe as Function of Look-back Interval",
+      xlab="look-back (months)", ylab="Sharpe")
```

Optimal Weighted Momentum Strategy for Stocks



```
> # Calculate best pnls of momentum strategy
> whichmax <- which.max(sharper)
> lookbv[whichmax]
> pnls <- pnll[[whichmax]]
> # Add initial startup interval to the momentum returns
> pnls <- c(retew[endd[1]:endd[3]], pnls)
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, 0.5*(retew + pnls))
> wealthv <- xts::xts(wealthv, order.by=datev)
> colnames(wealthv) <- c("EqualWeight", "Momentum", "Combined")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot dygraph of stock index and momentum strategy
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Weighted Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
```

Momentum Strategy With Daily Rebalancing

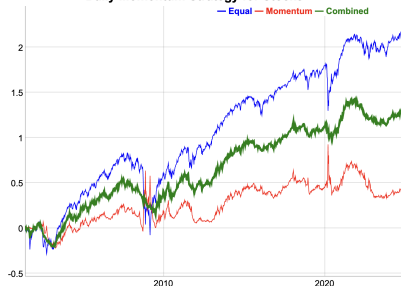
In a momentum strategy with *daily rebalancing*, the weights are updated every day and the portfolio is rebalanced accordingly.

The momentum strategy with *daily rebalancing* performs worse than with *monthly rebalancing* because of the daily variance of the weights.

A momentum strategy with *daily rebalancing* requires more computations so compiled C++ functions must be used instead of `apply()` loops.

The functions `HighFreq::run_mean()` and `HighFreq::run_var()` calculate the trailing mean and variance by recursively updating the past estimates with the new values, using the decay factor λ .

Daily Momentum Strategy for Stocks



```
> # Calculate the trailing average returns and variance using C++
> lambdaf <- 0.99
> varm <- HighFreq::run_var(retp, lambda=lambdaf)
> meanm <- varm[, 1:nstocks]
> varm <- varm[, (nstocks+1):(2*nstocks)]
> # Calculate the trailing Kelly ratios
> weightv <- meanm/varm
> weightv <- weightv/sqrt(rowSums(weightv^2, na.rm=TRUE))
> weightv <- rutils::lagit(weightv)
> # Calculate the momentum profits and losses
> pnls <- rowSums(weightv*retp, na.rm=TRUE)
> # Calculate the transaction costs
> bidask <- 0.0
> costv <- 0.5*bidask*rowSums(abs(rutils::diffit(weightv)), na.rm=TRUE)
> pnls <- (pnls - costv)

> # Scale the momentum volatility to the equal weight index
> volew <- sd(retew)
> pnls <- volew*pnls/sd(pnls)
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, 0.5*(retew + pnls))
> wealthv <- xts::xts(wealthv, datev)
> colnames(wealthv) <- c("EqualWeight", "Momentum", "Combined")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> # Plot dygraph of stock index and momentum strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Daily Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Daily Momentum Strategy Functional

The function `btmomdaily()` simulates a momentum strategy with *daily rebalancing*.

The decay factor λ determines the rate of decay of the weights applied to the returns, with smaller values of λ producing faster decay, giving more weight to recent returns, and vice versa.

If the argument `trend = -1` then it simulates a mean-reverting strategy (buys the worst performing stocks and sells the best performing).

Performing a *backtest* allows finding the optimal *momentum* (trading) strategy parameters, such as the *look-back interval*.

The function `btmomdaily()` can be used to find the best choice of *momentum strategy* parameters.

```
> # Define backtest functional for daily momentum strategy
> # If trend=(-1) then it backtests a mean reverting strategy
> btmomdaily <- function(retp, lambdaf=0.9, trend=1, bidask=0.0, ...
+   stopifnot("package:quantmod" %in% search() || require("quantmod"))
+   ## Calculate the trailing Kelly ratio
+   nstocks <- NCOL(retp)
+   varm <- HighFreq::run_var(retp, lambda=lambdaf)
+   meanm <- varm[, 1:nstocks]
+   varm <- varm[, (nstocks+1):(2*nstocks)]
+   varm[varm < 0.001] <- 0.001 ## Set volatility floor
+   weightv <- meanm/varm
+   weightv <- weightv/sqrt(rowSums(weightv^2, na.rm=TRUE))
+   weightv <- rutils::lagit(weightv)
+   ## Calculate the momentum profits and losses
+   pnls <- trend*rowSums(weightv*retp, na.rm=TRUE)
+   ## Calculate the transaction costs
+   costv <- 0.5*bidask*rowSums(abs(rutils::diffit(weightv)), na.rm=TRUE)
+   (pnls - costv)
+ } ## end btmomdaily
```

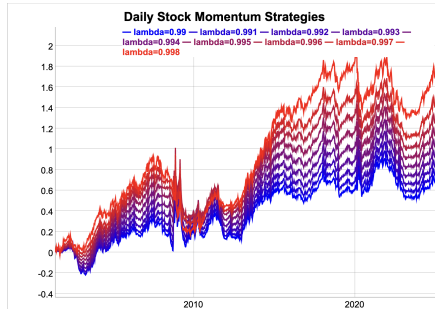
Multiple Daily Stock Momentum Strategies

Multiple daily momentum strategies can be backtested by calling the function `btmombdaily()` in a loop over a vector of λ parameters.

The best performing momentum strategies with *daily rebalancing* are with λ parameters close to 1.

The momentum strategies with *daily rebalancing* perform worse than with *monthly rebalancing* because of the daily variance of the weights.

```
> # Simulate multiple daily stock momentum strategies
> lambdav <- seq(0.99, 0.998, 0.001)
> pnls <- sapply(lambdav, btmombdaily, retp=retp)
> # Scale the momentum volatility to the equal weight index
> pnls <- apply(pnls, MARGIN=2, function(pnl) volew*pnl/sd(pnl))
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
> tail(pnls)
```



```
> # Plot dygraph of daily stock momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Daily Stock Momentum Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
> # Plot daily stock momentum strategies using quantmod
> themev <- chart_theme()
> themev$col$line.col <-
+   colorRampPalette(c("blue", "red"))(NCOL(pnls))
> quantmod::chart_Series(cumsum(pnls)[endw],
+   theme=themev, name="Daily Stock Momentum Strategies")
> legend("bottomleft", legend=colnames(pnls),
+   inset=0.02, bg="white", cex=0.7, lwd=rep(6, NCOL(retp)),
+   col=themev$col$line.col, bty="n")
```

Daily Momentum Strategy with Holding Period

The daily ETF momentum strategy can be improved by introducing a *holding period* for the portfolio, in order to reduce the variance of the portfolio weights.

Instead of holding the portfolio for only one day, it's held for several days and gradually liquidated. So that many past momentum portfolios are held at the same time.

This is equivalent to averaging the portfolio weights over the past.

The best length of the *holding period* depends on the *bias-variance tradeoff*.

If the *holding period* is too short then the weights have too much day-over-day *variance*.

If the *holding period* is too long then the weights have too much *bias* (they are stale).

The decay factor λ determines the length of the *holding period*. Smaller values of λ produce a faster decay corresponding to a shorter *holding period*, and vice versa.

The optimal value of the λ parameter can be determined by cross-validation (backtesting).

The function `btmmdailyhold()` simulates a momentum strategy with *daily rebalancing* with a holding period.

```
> # Define backtest functional for daily momentum strategy
> # If trend=(-1) then it backtests a mean reverting strategy
> btmmdailyhold <- function(retp, lambdaf=0.9, trend=1, bidask=0.0)
+   stopifnot("package:quantmod" %in% search() || require("quantmod"))
+   ## Calculate the trailing Kelly ratio
+   nstocks <- NCOL(retp)
+   varm <- HighFreq::run_var(retp, lambda=lambdaf)
+   meanm <- varm[, 1:nstocks]
+   varm <- varm[, (nstocks+1):(2*nstocks)]
+   varm[varm < 0.001] <- 0.001 ## Set volatility floor
+   weightv <- meanm/varm
+   weightv <- weightv/sqrt(rowSums(weightv^2, na.rm=TRUE))
+   ## Average the past weights
+   weightv <- HighFreq::run_mean(weightv, lambda=lambdaf)
+   weightv <- rutils::lagit(weightv)
+   ## Calculate the momentum profits and losses
+   pnls <- trend*rowSums(weightv*retp, na.rm=TRUE)
+   ## Calculate the transaction costs
+   costv <- 0.5*bidask*rowSums(abs(rutils::diffit(weightv)), na.rm=TRUE)
+   pnls <- (pnls - costv)
+   pnls[1:21] <- 0 ## Set the warmup PnLs to zero
+   return(pnls)
+ } ## end btmmdailyhold
```

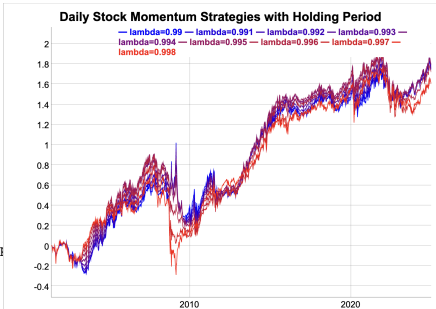
Multiple Daily Momentum Strategies With Holding Period

Multiple daily momentum strategies can be backtested by calling the function `btmcmdaily()` in a loop over a vector of λ parameters (holding periods).

The daily momentum strategies with a holding period perform better than with daily rebalancing.

The reason is that a longer holding period averages the weights and reduces their variance. But this also increases their bias, so there's an optimal holding period for an optimal bias-variance tradeoff.

```
> # Simulate multiple daily stock momentum strategies with holding
> lambdav <- seq(0.99, 0.998, 0.001)
> pnls <- sapply(lambdav, btmcmdailyhold, retp=retp)
> # Scale the momentum volatility to the equal weight index
> pnls <- apply(pnls, MARGIN=2, function(pnl) volew*pnl/sd(pnl))
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
```



```
> # dygraph of daily stock momentum strategies with holding period
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Daily Stock Momentum Strategies with Holding Period") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=500)
> # Plot of daily stock momentum strategies with holding period
> themev <- chart_theme()
> themev$col$line.col <-
+   colorRampPalette(c("blue", "red"))(NCOL(pnls))
> quantmod::chart_Series(cumsum(pnls)[endw],
+   theme=themev, name="Daily Stock Momentum Strategies with Holding
> legend("bottomleft", legend=colnames(pnls),
+   inset=0.02, bg="white", cex=0.7, lwd=rep(6, NCOL(retp)),
+   col=themev$col$line.col, bty="n")
```

Optimal Momentum Strategy With Holding Period

The daily momentum strategies with a holding period perform better than with daily rebalancing.

The reason is that a longer holding period averages the weights and reduces their variance. But this also increases their bias, so there's an optimal holding period for an optimal bias-variance tradeoff.

```
> # Calculate best pnls of momentum strategy
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> whichmax <- which.max(sharper)
> lambdav[whichmax]
> pnls <- pnls[, whichmax]
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, 0.5*(retew + pnls))
> colnames(wealthv) <- c("EqualWeight", "Momentum", "Combined")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



```
> # Plot dygraph of stock index and momentum strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Daily Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Mean Reverting Stock Momentum Strategies

Multiple *mean reverting* stock momentum strategies can be backtested by calling the function `btmomdaily()` in a loop over a vector of λ parameters.

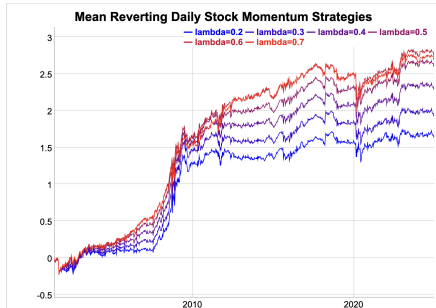
If the argument `trend = -1` then the function `btmomdaily()` simulates a mean-reverting strategy (buys the worst performing stocks and sells the best performing).

The *mean reverting* momentum strategies for the stock constituents perform the best for small λ parameters.

The *mean reverting* momentum strategies had their best performance prior to and during the 2008 financial crisis.

This simulation doesn't account for transaction costs, which could erase all profits if market orders were used for trade executions. But the strategy could be profitable if limit orders were used for trade executions.

```
> # Perform sapply loop over lambdav
> lambdav <- seq(0.2, 0.7, 0.1)
> pnls <- sapply(lambdav, btmomdaily, retp=retp, trend=(-1))
> # Scale the momentum volatility to the equal weight index
> pnls <- apply(pnls, MARGIN=2, function(pnl) volew*pnl/sd(pnl))
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
```



```
> # Plot dygraph of mean reverting daily stock momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Mean Reverting Daily Stock Momentum Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=400)
> # Plot mean reverting daily stock momentum strategies using quantmod
> themev <- chart_theme()
> themev$col$line.col <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> quantmod::chart_Series(cumsum(pnls)[endw],
+   theme=themev, name="Mean Reverting Daily Stock Momentum Strategies")
> legend("topleft", legend=colnames(pnls),
+   inset=0.05, bg="white", cex=0.7, lwd=rep(6, NCOL(retp)),
+   col=themev$col$line.col, bty="n")
```

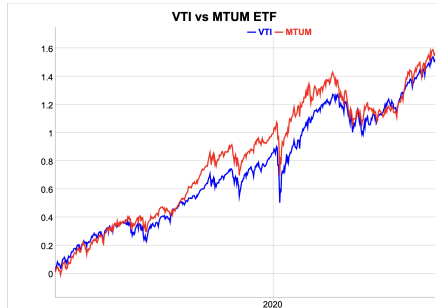
The MTUM Momentum ETF

The *MTUM* ETF is an actively managed ETF which follows a momentum strategy for stocks.

The *MTUM* ETF has a slightly higher absolute return than the *VTI* ETF, but it has a slightly lower Sharpe ratio.

The weak performance of the *MTUM* ETF demonstrates that it's difficult to implement a successful momentum strategy for individual stocks.

```
> # Calculate the scaled prices of VTI vs MTUM ETF
> wealthv <- na.omit(rutils::etfenv$prices[, c("VTI", "MTUM")])
> wealthv <- rutils::diffit(log(wealthv))
> colnames(wealthv) <- c("VTI", "MTUM")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
```



```
> # Plot of scaled prices of VTI vs MTUM ETF
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="VTI vs MTUM ETF") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(width=300)
```

PCA Portfolios

Principal Component Analysis (PCA) can be used to construct portfolios of stocks.

The principal components (*PCs*) are portfolios of stocks and can be traded directly as if they were single stocks.

PCA performs better if the stock returns are standardized (mean zero and unit variance).

PCA is equivalent to the *eigen decomposition* of either the correlation or the covariance matrix.

The function `eigen()` calculates the *eigenvectors* and *eigenvalues* of numeric matrices.

Performing *PCA* on returns with NA values is more complicated, so they are removed from the data.

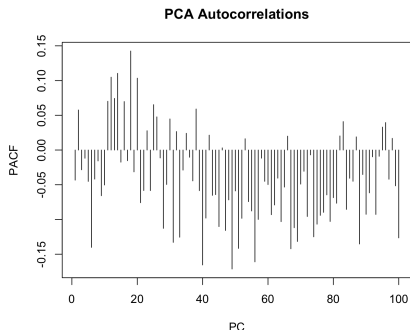
```
> # Select stocks with no NA values in their returns
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna == 0]
> nstocks <- NCOL(retp)
> retew <- rowMeans(retp, na.rm=TRUE)
> retew[1] <- 0
> volew <- sd(rowMeans(retp, na.rm=TRUE))
> # Calculate the standardized returns
> retsc <- lapply(retp, function(x) (x - mean(x))/sd(x))
> retsc <- do.call(cbind, retsc)
> # Calculate the PCA loadings
> covmat <- cov(retsc)
> pcad <- eigen(covmat)
> pcal <- pcad$vectors ## The PCA loadings
> rownames(pcal) <- colnames(retp)
> plot(sort(-pcal[, 1], decreasing=TRUE))
> plot(sort(pcal[, 2], decreasing=TRUE))
> round((t(pcal) %*% pcal)[1:5, 1:5], 4)
> # Calculate the PCA time series from the stock returns and the PCA
> retpca <- retp %*% pcal
> colnames(retpca) <- paste0("PC", 1:nstocks)
> round((t(retpca) %*% retpca)[1:5, 1:5], 4)
```

Autocorrelations of PCA Returns

The *Principal Component* portfolios (*PCs*) have either trending or mean-reverting characteristics.

The lower order *PCs* exhibit greater trending (positive autocorrelations) than individual stocks.

The higher order *PCs* exhibit greater mean reversion (negative autocorrelations) than the lower order ones.



```
> # Calculate the autocorrelations of the PCA time series
> pacv <- apply(retpca[, 1:100], 2, function(x)
+   sum(pacf(x, lag=10, plot=FALSE)$acf))
> plot(pacv, type="h", main="PCA Autocorrelations",
+   xlab="PC", ylab="PACF")
```

Momentum Strategy for PCA Portfolios

The momentum strategy performs better for *PCA* portfolios than for individual stocks for two reasons:

- The *PCA* returns are uncorrelated to each other,
- The lower order *PCA* returns have more positive autocorrelations than individual stocks.
- The higher order *PCA* returns have more negative autocorrelations than individual stocks.

If the stock returns are uncorrelated then the weights of the *maximum Sharpe* portfolio are proportional to the *Kelly ratios* (the returns divided by their variance):

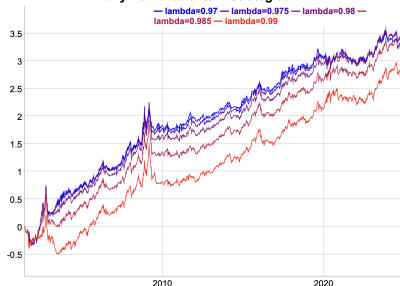
$$w_i = \mathbb{C}^{-1} \bar{r} = \frac{\bar{r}_i}{\sigma_i^2}$$

Where $\mathbb{C}$ is the covariance matrix of returns (which is diagonal in this case).

If the momentum weights are chosen to be the *Kelly ratios*, then the momentum strategy is equivalent to the *maximum Sharpe* optimal portfolio strategy.

```
> # Simulate daily PCA momentum strategies for multiple lambdabf pa:
> dimax <- 30
> lambdav <- seq(0.97, 0.99, 0.005)
> pnls <- mclapply(lambdav, btmomdailyhold, retp=retPCA[, 1:dimax])
> pnls <- lapply(pnls, function(pnl) volew*pnl/sd(pnl))
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
```

Daily PCA Momentum Strategies



```
> # Plot Sharpe ratios of momentum strategies
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> plot(x=lambdav, y=sharper, t="l",
+      main="PCA Momentum Sharpe as Function of Decay Factor",
+      xlab="lambdabf", ylab="Sharpe")
> # Plot dygraph of daily PCA momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> dygraphs::dygraph(cumsun(pnls)[endw],
+      main="Daily PCA Momentum Strategies") %>%
+      dyOptions(colors=colorv, strokeWidth=1) %>%
+      dyLegend(show="always", width=400)
```

Optimal PCA Momentum Strategy

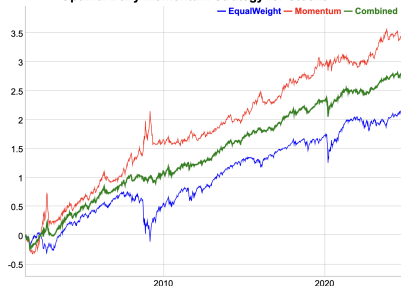
The *PCA* momentum strategy using only the lowest order *PCs* performs well when combined with the index.

But this is thanks to using the in-sample *PCs*.

The best performing *PCA* momentum strategy has a relatively small decay factor λ , so it's able to quickly adjust to changes in market direction.

```
> # Calculate best pnls of PCA momentum strategy
> whichmax <- which.max(Sharpe)
> lambdav[whichmax]
> pnls <- pnls[, whichmax]
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retew, pnls, 0.5*(retew + pnls))
> colnames(wealthv) <- c("EqualWeight", "Momentum", "Combined")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```

Optimal Daily Momentum Strategy for Stocks



```
> # Plot dygraph of stock index and PCA momentum strategy
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Daily Momentum Strategy for Stocks") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Mean Reverting PCA Momentum Strategy

The *mean reverting* momentum strategy buys the worst performing stocks and sells the best.

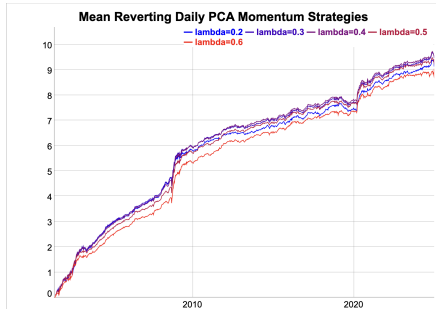
If the argument *trend* is set to -1 , then the function `btmomdailyhold()` simulates a mean-reverting strategy.

The *mean reverting* strategy performs well for the higher order PCs, but not the highest order, because those have very low volatilities.

The *mean reverting* strategies had their best performance in periods of high volatility, especially during the 2008 financial crisis.

The *mean reverting* strategy trades very frequently, so it's very sensitive to the transaction costs. If the transaction costs are too high then the strategy will lose money.

```
> # Simulate daily PCA momentum strategies for multiple lambda values
> lambdav <- seq(0.2, 0.6, 0.1)
> pnls <- mclapply(lambdav, btmomdailyhold, retp=retPCA[, (dimax+1):dimax],
+   trend=(-1), bidask=0.0001, mc.cores=ncores)
> pnls <- lapply(pnls, function(pnl) volew*pnl/sd(pnl))
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev)
```



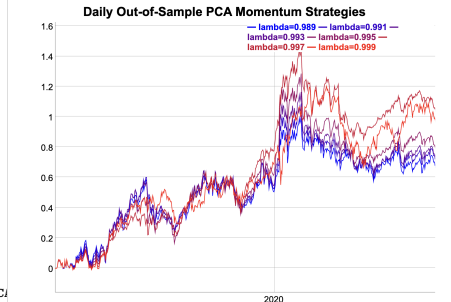
```
> # Plot Sharpe ratios of momentum strategies
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> plot(x=lambdav, y=sharper, t="l",
+   main="PCA Momentum Sharpe as Function of Decay Factor",
+   xlab="lambdav", ylab="Sharpe")
> # Plot dygraph of daily PCA momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Mean Reverting Daily PCA Momentum Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=400)
```

PCA Momentum Strategy Out-of-Sample

The principal component weights are calculated in-sample and applied out-of-sample.

The performance is much lower than in-sample, but it's still positive.

```
> # Define in-sample and out-of-sample intervals
> cutoff <- nrow %/% 2
> datev[cutoff]
> insample <- 1:cutoff
> outsample <- (cutoff + 1):nrow
> # Calculate the PCA loadings in-sample
> covmat <- cov(retsc[insample])
> pcd <- eigen(covmat)
> pcal <- pcd$vectors ## The PCA loadings
> rownames(pcal) <- colnames(retp)
> # Calculate the PCA time series from the stock returns and the PCA
> retpca <- retp %*% pcal
> colnames(retpca) <- paste0("PC", 1:nstocks)
>
> # Calculate the out-of-sample PCA time series
> retpca <- xts::xts(retpca[outsample, ], order.by=datev[outsample])
> # Simulate daily PCA momentum strategies for multiple lambda decay factors
> lambdav <- seq(0.989, 0.999, 0.002)
> pnls <- mclapply(lambdav, btmomdailyhold, retp=retpca[, 1:dimax])
> pnls <- lapply(pnls, function(pnl) volew*pnl/sd(pnl))
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev[outsample])
> # Plot Sharpe ratios of momentum strategies
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> plot(x=lambdav, y=sharper, t="l",
+      main="PCA Momentum Sharpe as Function of Decay Factor",
+      xlab="lambdaf", ylab="Sharpe")
```



```
> # Calculate a vector of weekly end points
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> # Plot dygraph of daily out-of-sample PCA momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Daily Out-of-Sample PCA Momentum Strategies") %>%
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300)
```

Mean Reverting PCA Momentum Strategy Out-of-Sample

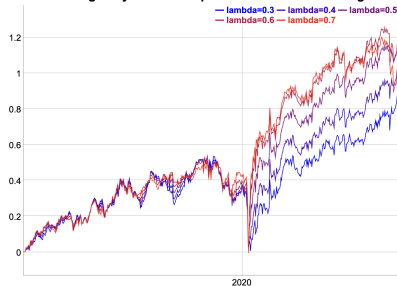
The principal component weights are calculated in-sample and applied out-of-sample.

The performance is much lower than in-sample, but it's still positive.

The *mean reverting* strategy trades very frequently, so it's very sensitive to the transaction costs. If the transaction costs are too high then the strategy will lose money.

```
> # Simulate daily PCA momentum strategies for multiple lambda factors
> lambdav <- seq(0.3, 0.7, 0.1)
> pnls <- mclapply(lambdav, btmomdailyhold,
+   retp=retpca[, (dimax+1):(NCOL(retpca)-dimax)],
+   trend=(-1), bidask=0.0001, mc.cores=ncores)
> pnls <- lapply(pnls, function(pnl) volew*pnl/sd(pnl))
> pnls <- do.call(cbind, pnls)
> colnames(pnls) <- paste0("lambda=", lambdav)
> pnls <- xts::xts(pnls, datev[outsample])
> # Plot Sharpe ratios of momentum strategies
> sharper <- sqrt(252)*sapply(pnls, function(pnl) mean(pnl)/sd(pnl))
> plot(x=lambdav, y=sharper, t="l",
+   main="PCA Momentum Sharpe as Function of Decay Factor",
+   xlab="lambdaf", ylab="Sharpe")
```

Mean Reverting Daily Out-of-Sample PCA Momentum Strategies



```
> # Calculate a vector of weekly end points
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> # Plot dygraph of daily S&P500 momentum strategies
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Mean Reverting Daily Out-of-Sample PCA Momentum Strategies",
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dyLegend(show="always", width=300))
```

Momentum Strategy for an *ETF* Portfolio

The performance of the momentum strategy depends on the length of the *look-back interval* used for calculating the past performance.

Research indicates that the optimal length of the *look-back interval* for momentum is about 4 to 10 months.

The dependence on the length of the *look-back interval* is an example of the *bias-variance tradeoff*.

If the *look-back interval* is too long, then the data has large *bias* because the distant past may have little relevance to today.

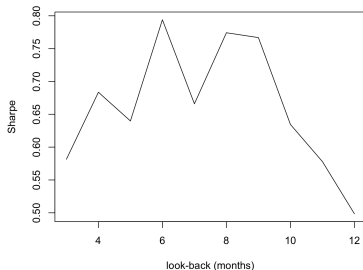
But if the *look-back interval* is too short, then there's not enough data, and estimates will have high *variance*.

Performing many *backtests* on multiple trading strategies risks identifying inherently unprofitable trading strategies as profitable, purely by chance (known as *p-value hacking*).

So *backtesting* just redefines the problem of finding (tuning) the optimal trading strategy parameters, into the problem of finding the optimal *backtest* (meta-model) parameters.

But the advantage of using the *backtest* meta-model is that it can reduce the number of parameters that need to be optimized.

Momentum Sharpe as Function of Look-back Interval



```
> # Extract ETF returns
> symbolv <- c("VTI", "IEF", "DBC")
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> datev <- zoo::index(retp)
> # Calculate a vector of monthly end points
> endd <- rutils::calc_endpoints(retp, interval="months")
> npts <- NROW(endd)
> # Perform backtests for vector of look-back intervals
> lookbv <- seq(3, 12, by=1)
> pnll <- lapply(lookbv, btomweight, retp=retp, endd=endd, objfun=
> sharper <- sqrt(252)*sapply(pnll, function(pnl) mean(pnl)/sd(pnl))
> # Plot Sharpe ratios of momentum strategies
> plot(x=lookbv, y=sharper, t="l",
+      main="Momentum Sharpe as Function of Look-back Interval",
+      xlab="look-back (months)", ylab="Sharpe")
```

Performance of Momentum Strategy for ETFs

The momentum strategy for ETFs produces a higher absolute return and also a higher Sharpe ratio than the static *All-Weather* portfolio.

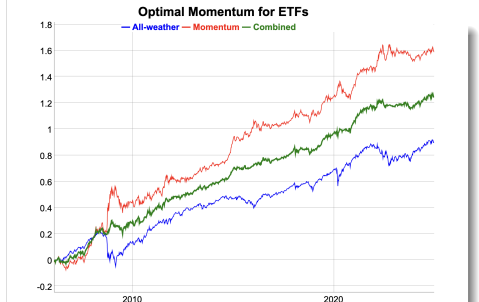
The momentum strategy for ETFs also has a very low correlation to the static *All-Weather* portfolio.

The momentum strategy works better for assets that are not correlated or are even anti-correlated.

The momentum strategy also works better for portfolios than for individual stocks because of risk diversification.

Portfolios of stocks can also be selected so that they are more autocorrelated - more trending - they have higher signal-to-noise ratios - larger Hurst exponents.

```
> # Calculate best pnls of momentum strategy
> whichmax <- which.max(sharper)
> lookbv[whichmax]
> pnls <- pnll[[whichmax]]
> retew <- rowMeans(retp)
> pnls <- c(retew[endd[1]:endd[3]], pnls)
> # Calculate returns of all-weather benchmark
> weightaw <- c(0.30, 0.55, 0.15)
> retaw <- retp %*% weightaw
> # Calculate the Sharpe and Sortino ratios
> wealthv <- cbind(retaw, pnls, 0.5*(retaw+pnls))
> wealthv <- xts::xts(wealthv, order.by=datev)
> colnames(wealthv) <- c("All-weather", "Momentum", "Combined")
> cor(wealthv)
> wealthv <- xts::xts(wealthv, order.by=datev)
> sharper <- sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
> sharper
```



```
> colnames(wealthv) <- paste(colnames(wealthv), round(sharper[1, ],
> # Plot dygraph of all-weather benchmark and momentum strategy
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Optimal Momentum for
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name=colnames(wealthv)[3], strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```

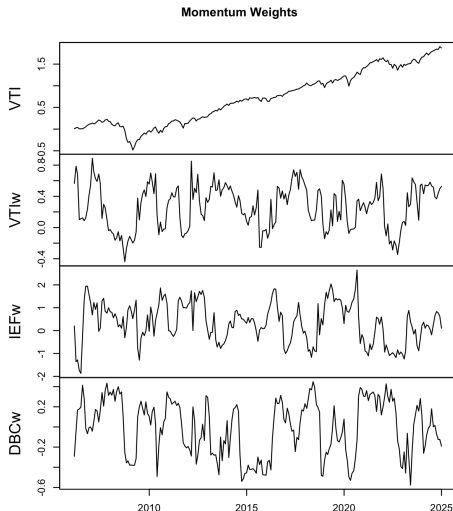
Time Series of Momentum Portfolio Weights

In momentum strategies, the portfolio weights are adjusted over time to be proportional to the past performance of the assets.

This way momentum strategies switch their weights to the best performing assets.

The weights are scaled to limit the portfolio *leverage* and its market *beta*.

```
> # Calculate the momentum weights
> lookb <- lookbv[whichmax]
> weightv <- lapply(2:npts, function(tday) {
+   ## Select the look-back returns
+   startp <- endd[max(1, tday-lookb)]
+   retis <- retp[startp:endd[tday], ]
+   ## Calculate weights proportional to performance
+   perfstat <- sapply(retis, objfun)
+   weightv <- drop(perfstat)
+   ## Scale weights so in-sample pnl volatility is same as equal w
+   pnls <- retis %*% weightv
+   weightv*sd(rowMeans(retis))/sd(pnls)
+ }) ## end lapply
> weightv <- rutils::do_call(rbind, weightv)
> # Plot of momentum weights
> retvti <- cumsum(retp$VTI)
> endw <- rutils::calc_endpoints(retp, interval="months")
> datav <- cbind(retvti[endw], weightv)
> colnames(datav) <- c("VTI", paste0(colnames(retp), "w"))
> zoo::plot.zoo(datav, xlab=NULL, main="Momentum Weights")
```

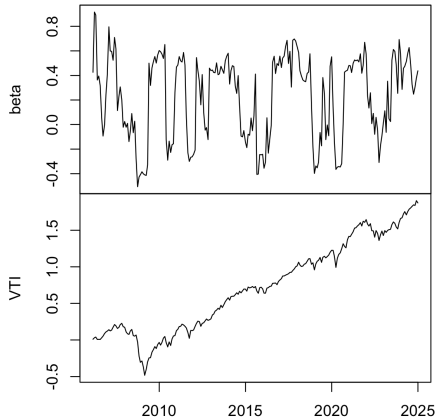


Momentum Strategy Market Beta

The momentum strategy market beta can be calculated by multiplying the *ETF* betas by the *ETF* portfolio weights.

```
> # Calculate ETF betas
> betetf <- sapply(retp, function(x) cov(retp$VTI, x)/var(retp$VTI))
> # Momentum beta is equal weights times ETF betas
> betam <- weightv %*% betetf
> betam <- xts::xts(betam, order.by=datev[endw])
> colnames(betam) <- "beta"
> datav <- cbind(betam, retvti[endw])
> zoo::plot.zoo(datav, main="Momentum Beta & VTI Returns", xlab="")
```

Momentum Beta & VTI Returns



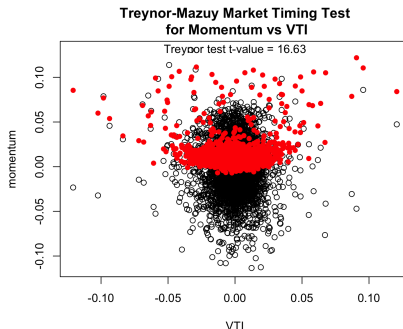
Momentum Strategy Market Timing Skill

Market timing skill is the ability to forecast the direction and magnitude of market returns.

Since the momentum strategy forecasts over one month intervals, we should also aggregate the returns to one month intervals.

The *Treynor-Mazuy* test shows that the momentum strategy has very significant *market timing* skill.

```
> # Aggregate the returns to monthly intervals
> retvti <- retp$VTI
> desm <- cbind(pnls, retvti, 0.5*(retvti+abs(retvti)), retvti^2)
> desm <- HighFreq::roll_sump(desm, lookb=22)
> colnames(desm) <- c("pnls", "VTI", "Merton", "Treynor")
> desm <- as.data.frame(desm)
> # Merton-Henriksson test
> regmod <- lm(pnls ~ VTI + Merton, data=desm); summary(regmod)
> # Treynor-Mazuy test
> regmod <- lm(pnls ~ VTI + Treynor, data=desm); summary(regmod)
> # Plot residual scatterplot
> resids <- regmod$residuals
> plot.default(x=retvti, y=resids, xlab="VTI", ylab="momentum")
> title(main="Treynor-Mazuy Market Timing Test\n for Momentum vs VTI", line=0.5)
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retvti
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retvti, y=fitv, pch=16, col="red")
> text(x=0.0, y=max(resids), paste("Treynor test t-value =", tvalue))
```



Skewness of Momentum Strategy Returns

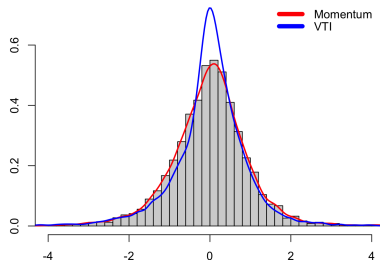
Most assets with *positive returns* suffer from *negative skewness*.

The momentum strategy returns have more positive skewness compared to the negative skewness of *VTI*.

The momentum strategy is a genuine *market anomaly*, because it has both positive returns and positive skewness.

```
> # Standardize the returns
> pnlsd <- (pnls-mean(pnls))/sd(pnls)
> retvti <- (retvti-mean(retvti))/sd(retvti)
> # Calculate skewness and kurtosis
> apply(cbind(pnlsd, retvti), 2, function(x)
+   sapply(c(skew=3, kurt=4),
+     function(e) sum(x^e))/NROW(retvti))
```

Momentum and VTI Return Distributions (standardized)



```
> # Calculate kernel density of VTI
> densvti <- density(retvti)
> # Plot histogram of momentum returns
> hist(pnlsd, breaks=80,
+   main="Momentum and VTI Return Distributions (standardized)",
+   xlim=c(-4, 4), ylim=range(densvti$y), xlab="", ylab="", freq=FALSE)
> # Draw kernel density of histogram
> lines(density(pnlsd), col='red', lwd=2)
> lines(densvti, col='blue', lwd=2)
> # Add legend
> legend("topright", inset=0.0, cex=1.0, title=NULL,
+   leg=c("Momentum", "VTI"), bty="n", y.intersp=0.7,
+   lwd=6, bg="white", col=c("red", "blue"))
```

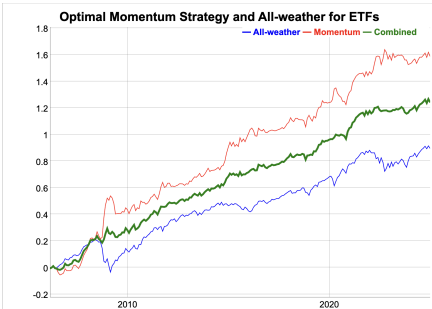
Combining Momentum with the All-Weather Portfolio

The momentum strategy has attractive returns compared to a static buy-and-hold strategy.

But the momentum strategy suffers from draw-downs called *momentum crashes*, especially after the market rallies from a sharp sell-off.

This suggests that combining the momentum strategy with a static buy-and-hold strategy can achieve significant diversification of risk.

```
> # Combine momentum strategy with all-weather
> wealthv <- cbind(retaw, pnls, 0.5*(pnls + retaw))
> colnames(wealthv) <- c("All-weather", "Momentum", "Combined")
> wealthv <- xts::xts(wealthv, datev)
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Calculate strategy correlations
> cor(wealthv)
```



```
> # Plot ETF momentum strategy combined with All-Weather
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimal Momentum Strategy and All-weather for ETFs") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Momentum Strategy for ETFs With Daily Rebalancing

In a momentum strategy with *daily rebalancing*, the weights are updated every day and the portfolio is rebalanced accordingly.

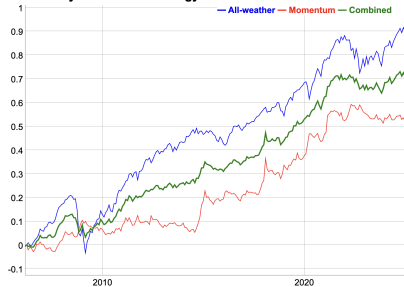
A momentum strategy with *daily rebalancing* requires more computations so compiled C++ functions must be used instead of `apply()` loops.

The momentum strategy with *daily rebalancing* performs worse than the strategy with *monthly rebalancing* because of the daily variance of the weights.

The advantage of the momentum strategy is that it's negatively correlated to the *All-Weather* returns, so when it's combined with it, it reduces the risk and increases the *Sharpe ratio*.

```
> # Calculate the EMA returns and variance
> varm <- HighFreq::run_var(retp, lambda=0.98)
> meanv <- varm[, 1:3]
> varm <- varm[, 4:6]
> # Calculate the trailing Kelly ratios
> weightv <- meanv/varm
> # weightv <- weightv/sqrt(rowSums(weightv^2))
> weightv <- rutils::lagit(weightv)
> # Calculate the momentum profits and losses
> pnls <- rowSums(weightv*retp)
> # Calculate the transaction costs
> bidask <- 0.0
> costv <- 0.5*bidask*rowSums(abs(rutils::diffit(weightv)))
> pnls <- (pnls - costv)
```

Daily Momentum Strategy for ETFs vs All-Weather



```
> # Scale the momentum volatility to all-weather
> pnls <- sd(retaw)*pnls/sd(pnls)
> # Calculate the wealth of momentum returns
> wealthv <- cbind(retaw, pnls, 0.5*(pnls + retaw))
> colnames(wealthv) <- c("All-weather", "Momentum", "Combined")
> wealthv <- xts::xts(wealthv, datev)
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> cor(wealthv)
> # Plot dygraph of the momentum strategy returns
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Daily Momentum Strategy for ETFs vs All-Weather") %>%
+   dyOptions(colors=c("blue", "red", "green"), strokeWidth=1) %>%
+   dySeries(name="Combined", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Backtesting of Strategies

Backtesting is the simulation of a trading or investment strategy on historical data.

Backtesting is performed by *training* the strategy (calibrating the model parameters) on the past in-sample data and *testing* it on future out-of-sample data.

Backtesting is a type of *cross-validation* applied to investment strategies.

Backtest simulations can be useful for determining the relative advantages of different strategy features and model parameters.

Backtesting can be used to identify strategies that have little chance of being profitable.

Because if a strategy is not profitable in backtesting then it has little chance of being profitable in live trading.

Backtesting can be used to identify strategies that have a better chance of being profitable than others.

Backtesting can be used to rank strategies from the most promising to the least promising.

But backtesting cannot ensure that a strategy will be profitable in live trading, just because it is profitable in simulation.

Backtesting of Momentum Strategies

Backtesting is performed by *training* the strategy (calibrating the model parameters) on the past in-sample data and *testing* it on future out-of-sample data.

The *momentum* portfolio weights are calculated using the past stock returns, which are determined by the length of the *look-back* interval.

The *momentum* strategy returns (pnls) are calculated by multiplying the out-of-sample stock returns times the forecast portfolio weights.

The *transaction costs* are equal to half the *bid-ask spread* δ times the absolute value of the traded dollar amounts of the *risky assets*.

The dependence on the length of the *look-back interval* is an example of the *bias-variance tradeoff*.

If the *look-back interval* is too long, then the portfolio weights have large *bias* because the distant past may have little relevance to today.

But if the *look-back interval* is too short, then there's not enough data, and the portfolio weights will have high *variance*.

Backtesting can help to determine the optimal length of the *look-back interval*.

Limitations of Backtest Simulations

The simulated strategy pnl's from backtesting may not be realistic for several reasons:

- Transaction costs (broker commissions, bidask spread, market impact).
- Costs of stock borrowing (stocks must be borrowed in order to short them).
- Limits on stock borrowing and shorting (stocks may not be available for borrow, or they may be prohibited from shorting).

So the backtest simulations should be thought of as experiments to explore which features and parameters have better potential for profit.

But the backtest simulations cannot guarantee future profits.

In addition, performing many *backtests* on multiple trading strategies risks identifying inherently unprofitable strategies as profitable, purely by chance (known as *p-value hacking*).

The Minimum Variance Portfolio

The portfolio variance is equal to: $\mathbf{w}^T \mathbb{C} \mathbf{w}$, where $\mathbb{C}$ is the covariance matrix of returns.

If the portfolio weights $\mathbf{w}$ are subject to *linear* constraints: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$, then the weights that minimize the portfolio variance can be found by minimizing the *Lagrangian*:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C} \mathbf{w} - \lambda (\mathbf{w}^T \mathbf{1} - 1)$$

Where λ is a *Lagrange multiplier*.

The derivative of a scalar variable with respect to a vector variable is a vector, for example:

$$d_w[\mathbf{w}^T \mathbf{1}] = d_w[\mathbf{1}^T \mathbf{w}] = \mathbf{1}^T$$

$$d_w[\mathbf{w}^T \mathbf{r}] = d_w[\mathbf{r}^T \mathbf{w}] = \mathbf{r}^T$$

$$d_w[\mathbf{w}^T \mathbb{C} \mathbf{w}] = \mathbf{w}^T \mathbb{C} + \mathbf{w}^T \mathbb{C}^T$$

Where $\mathbf{1}$ is the unit vector, and $\mathbf{w}^T \mathbf{1} = \mathbf{1}^T \mathbf{w} = \sum_{i=1}^n x_i$

The derivative of the *Lagrangian* $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$d_w \mathcal{L} = 2\mathbf{w}^T \mathbb{C} - \lambda \mathbf{1}^T$$

By setting the derivative to zero we find $\mathbf{w}$ equal to:

$$\mathbf{w} = \frac{1}{2} \lambda \mathbb{C}^{-1} \mathbf{1}$$

By multiplying the above from the left by $\mathbf{1}^T$, and using $\mathbf{w}^T \mathbf{1} = 1$, we find λ to be equal to:

$$\lambda = \frac{2}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}}$$

And finally the portfolio weights are then equal to:

$$\mathbf{w} = \frac{\mathbb{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}}$$

If the portfolio weights are subject to *quadratic* constraints: $\mathbf{w}^T \mathbf{w} = 1$ then the minimum variance weights are equal to the highest order *principal component* (with the smallest eigenvalue) of the covariance matrix $\mathbb{C}$.

Returns and Variance of the *Minimum Variance Portfolio*

The stock weights of the *minimum variance* portfolio under the constraint $\mathbf{w}^T \mathbf{1} = 1$ can be calculated using the inverse of the covariance matrix:

$$\mathbf{w} = \frac{\mathbf{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}}$$

The daily returns of the *minimum variance* portfolio are equal to:

$$\mathbf{r}_{mv} = \frac{\mathbf{r}^T \mathbf{C}^{-1} \mathbf{1}}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}} = \frac{\mathbf{r}^T \mathbf{C}^{-1} \mathbf{1}}{c_{11}}$$

Where $\mathbf{r}$ are the daily stock returns, and $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$.

The variance of the *minimum variance* portfolio is equal to:

$$\sigma_{mv}^2 = \mathbf{w}^T \mathbf{C} \mathbf{w} = \frac{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{C} \mathbf{C}^{-1} \mathbf{1}}{(\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1})^2} = \frac{1}{\mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}} = \frac{1}{c_{11}}$$

The function `solve()` solves systems of linear equations, and also inverts square matrices.

The `%*` operator performs *inner (scalar)* multiplication of vectors and matrices.

Inner multiplication multiplies the rows of one matrix with the columns of another matrix.

The function `drop()` removes any extra dimensions of length *one*.

```
> # Calculate daily ETF returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> # Calculate covariance matrix of returns and its inverse
> covmat <- cov(retp)
> covinv <- solve(a=covmat)
> unitv <- rep(1, nstocks)
> # Calculate the minimum variance weights
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> weightmv <- drop(covinv %*% unitv/c11)
> # Calculate the daily minvar portfolio returns in two ways
> retmv <- (retp %*% weightmv)
> all.equal(retmv, (retp %*% covinv %*% unitv)/c11)
> # Calculate the minimum variance in three ways
> all.equal(var(retmv),
+   t(weightmv) %*% covmat %*% weightmv,
+   1/(t(unitv) %*% covinv %*% unitv))
```

The Efficient Portfolios

A portfolio which has the smallest variance, given a target return, is an *efficient portfolio*.

The efficient portfolio weights have two constraints: the sum of portfolio weights $\mathbf{w}$ is equal to 1: $\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$, and the mean portfolio return is equal to the target return r_t : $\mathbf{w}^T \bar{\mathbf{r}} = \sum_{i=1}^n w_i \bar{r}_i = r_t$.

Where $\bar{\mathbf{r}}$ are the mean stock returns.

The stock weights that minimize the portfolio variance under these constraints can be found by minimizing the *Lagrangian*:

$$\mathcal{L} = \mathbf{w}^T \mathbf{C} \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} - 1) - \lambda_2 (\mathbf{w}^T \bar{\mathbf{r}} - r_t)$$

Where λ_1 and λ_2 are the *Lagrange multipliers*.

The derivative of the *Lagrangian* $\mathcal{L}$ with respect to $\mathbf{w}$ is given by:

$$d_{\mathbf{w}} \mathcal{L} = 2\mathbf{w}^T \mathbf{C} - \lambda_1 \mathbf{1}^T - \lambda_2 \bar{\mathbf{r}}^T$$

By setting the derivative to zero we obtain the efficient portfolio weights $\mathbf{w}$:

$$\mathbf{w} = \frac{1}{2} (\lambda_1 \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{C}^{-1} \bar{\mathbf{r}})$$

By multiplying the above from the left first by $\mathbf{1}^T$, and then by $\bar{\mathbf{r}}^T$, we obtain a system of two equations for λ_1 and λ_2 :

$$2\mathbf{1}^T \mathbf{w} = \lambda_1 \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}} = 2$$

$$2\bar{\mathbf{r}}^T \mathbf{w} = \lambda_1 \bar{\mathbf{r}}^T \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}} = 2r_t$$

The above can be written in matrix notation as:

$$\begin{bmatrix} \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1} & \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}} \\ \bar{\mathbf{r}}^T \mathbf{C}^{-1} \mathbf{1} & \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2r_t \end{bmatrix}$$

Or:

$$\begin{bmatrix} c_{11} & c_{r1} \\ c_{r1} & c_{rr} \end{bmatrix} \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} = \mathbb{F} \lambda = 2 \begin{bmatrix} 1 \\ r_t \end{bmatrix} = 2\mathbf{u}$$

With $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$, $c_{r1} = \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $c_{rr} = \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $\lambda = \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix}$, $\mathbf{u} = \begin{bmatrix} 1 \\ r_t \end{bmatrix}$, and $\mathbb{F} = \mathbf{u}^T \mathbf{C}^{-1} \mathbf{u} = \begin{bmatrix} c_{11} & c_{r1} \\ c_{r1} & c_{rr} \end{bmatrix}$.

The *Lagrange multipliers* can be solved as:

$$\lambda = 2\mathbb{F}^{-1} \mathbf{u}$$

The Efficient Portfolio Weights

The efficient portfolio weights $\mathbf{w}$ can now be solved as:

$$\mathbf{w} = \frac{1}{2}(\lambda_1 \mathbf{C}^{-1} \mathbf{1} + \lambda_2 \mathbf{C}^{-1} \bar{\mathbf{r}}) =$$

$$\frac{1}{2} \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \lambda = \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \mathbb{F}^{-1} \mathbf{u} =$$

$$\frac{1}{\det \mathbb{F}} \begin{bmatrix} \mathbf{C}^{-1} \mathbf{1} \\ \mathbf{C}^{-1} \bar{\mathbf{r}} \end{bmatrix}^T \begin{bmatrix} c_{rr} & -c_{r1} \\ -c_{r1} & c_{11} \end{bmatrix} \begin{bmatrix} 1 \\ r_t \end{bmatrix} =$$

$$\frac{(c_{rr} - c_{r1} r_t) \mathbf{C}^{-1} \mathbf{1} + (c_{11} r_t - c_{r1}) \mathbf{C}^{-1} \bar{\mathbf{r}}}{\det \mathbb{F}}$$

With $c_{11} = \mathbf{1}^T \mathbf{C}^{-1} \mathbf{1}$, $c_{r1} = \mathbf{1}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$, $c_{rr} = \bar{\mathbf{r}}^T \mathbf{C}^{-1} \bar{\mathbf{r}}$.
And $\det \mathbb{F} = c_{11} c_{rr} - c_{r1}^2$ is the determinant of the matrix $\mathbb{F}$.

The above formula shows that the efficient portfolio weights are a linear function of the target return.

Therefore a convex sum of two efficient portfolio weights: $\mathbf{w} = \alpha \mathbf{w}_1 + (1 - \alpha) \mathbf{w}_2$, are also the weights of an *efficient portfolio*, with target return equal to:
 $r_t = \alpha r_1 + (1 - \alpha) r_2$

```
> # Calculate vector of mean returns
> retm <- colMeans(retp)
> # Specify the target return
> retarg <- 1.5*mean(retp)
> # Products of inverse with mean returns and unit vector
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> cr1 <- drop(t(unitv) %*% covinv %*% retm)
> crr <- drop(t(retm) %*% covinv %*% retm)
> fmat <- matrix(c(c11, cr1, cr1, crr), nc=2)
> # Solve for the Lagrange multipliers
> lagm <- solve(a=fmat, b=c(2, 2*retarg))
> # Calculate the efficient portfolio weights
> weightv <- 0.5*drop(covinv %*% cbind(unitv, retm) %*% lagm)
> # Calculate constraints
> all.equal(1, sum(weightv))
> all.equal(retarg, sum(retm*weightv))
```

Variance of the *Efficient Portfolios*

The *efficient portfolio* variance is equal to:

$$\sigma^2 = \mathbf{w}^T \mathbb{C} \mathbf{w} = \frac{1}{4} \lambda^T \mathbb{F} \lambda = \mathbf{u}^T \mathbb{F}^{-1} \mathbf{u} =$$

$$\frac{1}{\det \mathbb{F}} \begin{bmatrix} 1 \\ r_t \end{bmatrix}^T \begin{bmatrix} c_{rr} & -c_{r1} \\ -c_{r1} & c_{11} \end{bmatrix} \begin{bmatrix} 1 \\ r_t \end{bmatrix} =$$

$$\frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{\det \mathbb{F}}$$

The above formula shows that the variance of the *efficient portfolios* is a *parabola* with respect to the target return r_t .

The vertex of the *parabola* is the minimum variance portfolio: $r_{mv} = c_{r1}/c_{11} = \mathbf{1}^T \mathbb{C}^{-1} \mathbf{r} / \mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}$ and $\sigma_{mv}^2 = 1/c_{11} = 1/\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1}$.

The *efficient portfolio* variance can be expressed in terms of the difference $\Delta_r = r_t - r_{mv}$ as:

$$\sigma^2 = \frac{\Delta_r^2 + \det \mathbb{F}}{c_{11} \det \mathbb{F}}$$

So that if $\Delta_r = 0$ then $\sigma^2 = 1/c_{11}$.

Where $\det \mathbb{F} = c_{11}c_{rr} - c_{r1}^2$ is the determinant of the matrix $\mathbb{F}$.

```
> # Calculate the efficient portfolio returns
> reteff <- drop(retp %*% weightv)
> reteffm <- mean(reteff)
> all.equal(reteffm, retarg)
> # Calculate the efficient portfolio variance in three ways
> uu <- c(1, retarg)
> finv <- solve(fmat)
> detf <- (c11*crr-cr1^2) ## det(fmat)
> all.equal(var(reteff),
+ drop(t(uu) %*% finv %*% uu),
+ (c11*reteffm^2-2*cr1*reteffm+crr)/detf)
```

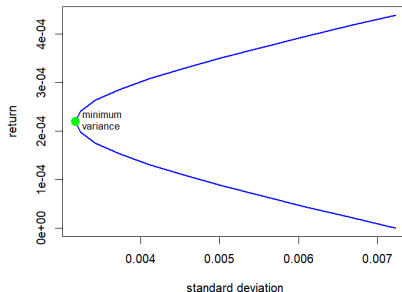
The Efficient Frontier

The *efficient frontier* is the set of *efficient portfolios*, that have the lowest risk (standard deviation) for the given level of return.

The *efficient frontier* is the plot of the target returns r_t and the standard deviations of the *efficient portfolios*, which is a *hyperbola*.

```
> # Calculate the daily and mean minvar portfolio returns
> c11 <- drop(t(unitv) %*% covinv %*% unitv)
> weightv <- drop(covinv %*% unitv/c11)
> retmv <- (retp %*% weightv)
> retmvm <- sum(weightv*retp)
> # Calculate the minimum variance
> varmv <- 1/c11
> stdevmv <- sqrt(varmv)
> # Calculate efficient frontier from target returns
> retargv <- retmvm*(1+seq(from=(-1), to=1, by=0.1))
> stdevs <- sapply(retargv, function(rett) {
+   uu <- c(1, rett)
+   sqrt(drop(t(uu) %*% finv %*% uu))
+ }) ## end sapply
```

Efficient Frontier and Minimum Variance Portfolio



```
> # Plot the efficient frontier
> plot(x=stdevs, y=retargv, t="l", col="blue", lwd=2,
+   main="Efficient Frontier and Minimum Variance Portfolio",
+   xlab="standard deviation", ylab="return")
> points(x=stdevmv, y=retmvm, col="green", lwd=6)
> text(x=stdevmv, y=retmvm, labels="minimum \nvariance",
+   pos=4, cex=0.8)
```

The Tangent Line and the Risk-free Rate

The *tangent* line connects the risk-free point ($\sigma = 0, r = r_f$) with a single tangent point on the *efficient frontier*.

A *tangent* line can be drawn at every point on the *efficient frontier*.

The slope β of the *tangent* line can be calculated by differentiating the efficient portfolio variance σ^2 by the target return r_t :

$$\begin{aligned}\frac{d\sigma^2}{dr_t} &= 2\sigma \frac{d\sigma}{dr_t} = \frac{2c_{11}r_t - 2c_{r1}}{\det \mathbb{F}} \\ \frac{d\sigma}{dr_t} &= \frac{c_{11}r_t - c_{r1}}{\sigma \det \mathbb{F}} \\ \beta &= \frac{\sigma \det \mathbb{F}}{c_{11}r_t - c_{r1}}\end{aligned}$$

The *tangent* line connects the *tangent* point on the *efficient frontier* with a *risk-free* rate r_f .

The *risk-free* rate r_f can be calculated as the intercept of the tangent line:

$$\begin{aligned}r_f &= r_t - \sigma \beta = r_t - \frac{\sigma^2 \det \mathbb{F}}{c_{11}r_t - c_{r1}} = \\ r_t - \frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{\det \mathbb{F}} \frac{\det \mathbb{F}}{c_{11}r_t - c_{r1}} &= \\ r_t - \frac{c_{11}r_t^2 - 2c_{r1}r_t + c_{rr}}{c_{11}r_t - c_{r1}} &= \frac{c_{r1}r_t - c_{rr}}{c_{11}r_t - c_{r1}}\end{aligned}$$

```
> # Calculate standard deviation of efficient portfolio
> uu <- c(1, retarg)
> stdeveff <- sqrt(drop(t(uu) %*% finv %*% uu))
> # Calculate the slope of the tangent line
> detf <- (c11*crr - cr1^2) ## det(fmat)
> sharper <- (stdeveff*detf)/(c11*retarg - cr1)
> # Calculate the risk-free rate as intercept of the tangent line
> raterf <- retarg - sharper*stdeveff
> # Calculate the risk-free rate from target return
> all.equal(raterf,
+ (retarg*cr1 - crr)/(retarg*c11 - cr1))
```

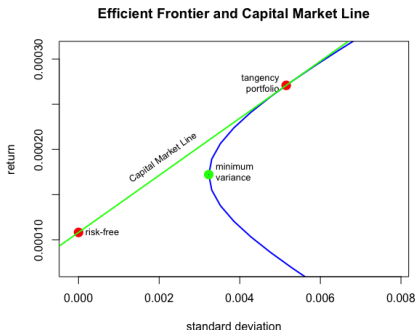
The Capital Market Line

The *Capital Market Line* (CML) is the tangent line connecting the risk-free point ($\sigma = 0, r = r_f$) with a single tangent point on the *efficient frontier*.

The *tangency portfolio* is the *efficient portfolio* at the tangent point corresponding to the given *risk-free rate*.

Each value of the *risk-free rate* r_f corresponds to a unique *tangency portfolio*.

For a given *risk-free rate* r_f , the *tangency portfolio* has the highest *Sharpe ratio* among all the *efficient portfolios*.



```
> # Plot efficient frontier
> aspectr <- 1.0*max(stdevs)/diff(range(retargv)) ## Aspect ratio
> plot(x=stdevs, y=retargv, t="l", col="blue", lwd=2, asp=aspectr,
+      xlim=c(0.4, 0.6)*max(stdevs), ylim=c(0.2, 0.9)*max(retargv),
+      main="Efficient Frontier and Capital Market Line",
+      xlab="standard deviation", ylab="return")
> # Plot the minimum variance portfolio
> points(x=stdevmv, y=retmv, col="green", lwd=6)
> text(x=stdevmv, y=retmv, labels="minimum \nvariance",
+      pos=4, cex=0.8)
```

```
> # Plot the tangent portfolio
> points(x=stdeveff, y=retarg, col="red", lwd=6)
> text(x=stdeveff, y=retarg, labels="tangency\nportfolio", pos=2, cex=0.8)
> # Plot the risk-free point
> points(x=0, y=raterf, col="red", lwd=6)
> text(x=0, y=raterf, labels="risk-free", pos=4, cex=0.8)
> # Plot the tangent line
> abline(a=raterf, b=sharper, lwd=2, col="green")
> text(x=0.6*stdev, y=0.8*retarg,
+      labels="Capital Market Line", pos=2, cex=0.8,
+      srt=180/pi*atan(aspectr*sharper))
```

The Capital Market Line Portfolios

The points on the *Capital Market Line* represent portfolios consisting of the *tangency portfolio* and the *risk-free asset* (bond).

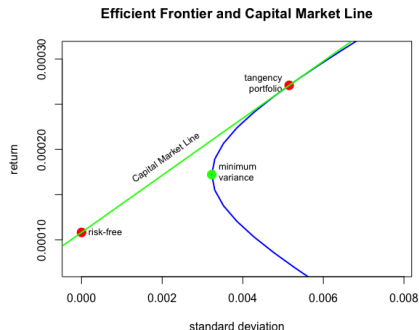
The *Capital Market Line* represents delevered and levered portfolios, consisting of the *tangency portfolio* combined with the *risk-free asset* (bond).

The *CML* portfolios have weights proportional to the tangency portfolio weights.

The *CML* portfolios above the tangent point are levered with respect to the *tangency portfolio* through borrowing at the *risk-free rate* r_f . Their weights are equal to the tangency portfolio weights multiplied by a factor greater than 1.

The *CML* portfolios below the tangent point are delevered with respect to the *tangency portfolio* through investing at the *risk-free rate* r_f . Their weights are equal to the tangency portfolio weights multiplied by a factor less than 1.

All the *CML* portfolios have the same *Sharpe ratio*.



Maximum Sharpe Portfolio Weights

The *Sharpe* ratio is equal to the ratio of excess returns divided by the portfolio standard deviation:

$$SR = \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma}$$

Where $\boldsymbol{\mu} = \bar{\mathbf{r}} - r_f$ is the vector of mean excess returns (in excess of the risk-free rate r_f), $\mathbf{w}$ is the vector of portfolio weights, and $\sigma = \sqrt{\mathbf{w}^T \mathbb{C} \mathbf{w}}$, where $\mathbb{C}$ is the covariance matrix of returns.

We can calculate the *maximum Sharpe* portfolio weights by setting the derivative of the *Sharpe* ratio with respect to the weights, to zero:

$$d_w SR = \frac{1}{\sigma} (\boldsymbol{\mu}^T - \frac{(\mathbf{w}^T \boldsymbol{\mu})(\mathbf{w}^T \mathbb{C})}{\sigma^2}) = 0$$

We then get:

$$(\mathbf{w}^T \mathbb{C} \mathbf{w}) \boldsymbol{\mu} = (\mathbf{w}^T \boldsymbol{\mu}) \mathbb{C} \mathbf{w}$$

We can multiply the above equation by $\mathbb{C}^{-1}$ to get:

$$\mathbf{w} = \frac{\mathbf{w}^T \mathbb{C} \mathbf{w}}{\mathbf{w}^T \boldsymbol{\mu}} \mathbb{C}^{-1} \boldsymbol{\mu}$$

We can finally rescale the weights so that they satisfy the linear constraint $\mathbf{w}^T \mathbf{1} = 1$:

$$\mathbf{w} = \frac{\mathbb{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}^{-1} \boldsymbol{\mu}}$$

These are the weights of the *maximum Sharpe* portfolio, with the vector of mean excess returns equal to $\boldsymbol{\mu}$, and the covariance matrix equal to $\mathbb{C}$.

The *maximum Sharpe* portfolio is an *efficient portfolio*, and so its mean return is equal to some target return r_t : $\bar{\mathbf{r}}^T \mathbf{w} = \sum_{i=1}^n w_i r_i = r_t$.

The mean return of the *maximum Sharpe* portfolio is equal to:

$$r_t = \bar{\mathbf{r}}^T \mathbf{w} = \frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}^{-1} \boldsymbol{\mu}} = \frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} (\bar{\mathbf{r}} - r_f)}{\mathbf{1}^T \mathbb{C}^{-1} (\bar{\mathbf{r}} - r_f)} =$$

$$\frac{\bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1} r_f - \bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1} \bar{\mathbf{r}}}{\mathbf{1}^T \mathbb{C}^{-1} \mathbf{1} r_f - \bar{\mathbf{r}}^T \mathbb{C}^{-1} \mathbf{1}} = \frac{c_{r1} r_f - c_{rr}}{c_{11} r_f - c_{r1}}$$

The above formula calculates the target return r_t from the risk-free rate r_f .

Returns and Variance of the Maximum Sharpe Portfolio

The *maximum Sharpe* portfolio weights depend on the value of the risk-free rate r_f :

$$\mathbf{w} = \frac{\mathbf{C}^{-1}\boldsymbol{\mu}}{\mathbf{1}^T\mathbf{C}^{-1}\boldsymbol{\mu}} = \frac{\mathbf{C}^{-1}(\bar{\mathbf{r}} - r_f)}{\mathbf{1}^T\mathbf{C}^{-1}(\bar{\mathbf{r}} - r_f)}$$

The mean return of the *maximum Sharpe* portfolio is equal to:

$$r_t = \bar{\mathbf{r}}^T \mathbf{w} = \frac{\bar{\mathbf{r}}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} = \frac{c_{r1} r_f - c_{rr}}{c_{11} r_f - c_{r1}}$$

The variance of the *maximum Sharpe* portfolio is equal to:

$$\begin{aligned} \sigma^2 &= \mathbf{w}^T \mathbf{C} \mathbf{w} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \mathbf{C} \mathbf{C}^{-1} \boldsymbol{\mu}}{(\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu})^2} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{(\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu})^2} = \\ &= \frac{(\bar{\mathbf{r}} - r_f)^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f)}{(\mathbf{1}^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f))^2} = \frac{c_{11} r_t^2 - 2c_{r1} r_t + c_{rr}}{\det \mathbb{F}} \end{aligned}$$

The above formula expresses the *maximum Sharpe* portfolio variance as a function of its mean return r_t .

The *maximum Sharpe* ratio is equal to:

$$\begin{aligned} SR &= \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma} = \frac{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} / \frac{\sqrt{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}}{\mathbf{1}^T \mathbf{C}^{-1} \boldsymbol{\mu}} = \\ &= \frac{\sqrt{\boldsymbol{\mu}^T \mathbf{C}^{-1} \boldsymbol{\mu}}}{\sqrt{(\bar{\mathbf{r}} - r_f)^T \mathbf{C}^{-1} (\bar{\mathbf{r}} - r_f)}} \end{aligned}$$

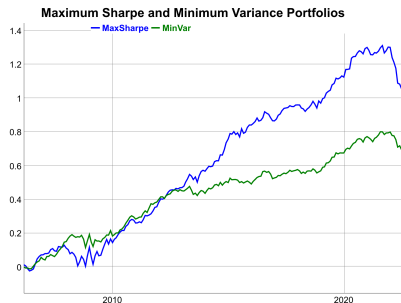
```
> # Calculate the mean excess returns
> raterf <- retarg - sharper*stdevdiff
> retx <- (retm - raterf)
> # Calculate the efficient portfolio weights
> weightv <- 0.5*drop(covinv %*% cbind(unitv, retm) %*% lagm)
> # Calculate the maximum Sharpe weights
> weightms <- drop(covinv %*% retx)/sum(covinv %*% retx)
> all.equal(weightv, weightms)
> # Calculate the maximum Sharpe mean return in two ways
> all.equal(sum(retm*weightv), (c11*raterf-crr)/(c11*raterf-cr1))
> # Calculate the maximum Sharpe daily returns
> retdd <- (retp %*% weightms)
> # Calculate the maximum Sharpe variance in four ways
> detf <- (c11*crr-cr1^2) ## det(fmat)
> all.equal(var(retdd),
+   t(weightv) %*% covmat %*% weightv,
+   (t(retx) %*% covinv %*% retx)/sum(covinv %*% retx)^2,
+   (c11*retarg^2-2*c11*retarg*crr)/detf)
> # Calculate the maximum Sharpe ratio
> sqrt(252)*sum(weightv*retx)/
+   sqrt(drop(t(weightv) %*% covmat %*% weightv))
> # Calculate the stock Sharpe ratios
> sqrt(252)*sapply((retp - raterf), function(x) mean(x)/sd(x))
```

Maximum Sharpe and Minimum Variance Performance

The *maximum Sharpe* and *Minimum Variance* portfolios are both *efficient portfolios*, with the lowest risk (standard deviation) for the given level of return.

The *maximum Sharpe* portfolio has both a higher Sharpe ratio and higher absolute returns.

```
> # Calculate optimal portfolio returns
> wealthv <- cbind(retp %*% weightms, retp %*% weightmv)
> wealthv <- xts::xts(wealthv, zoo::index(retp))
> colnames(wealthv) <- c("MaxSharpe", "MinVar")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   (mean(x)-raterf)/c(Sharpe=sd(x), Sortino=sd(x[x<0])))
> # Plot the log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Maximum Sharpe and Minimum Variance Portfolios") %>%
+   dyOptions(colors=c("blue", "green"), strokeWidth=2) %>%
+   dyLegend(show="always", width=500)
```



The Maximum Sharpe Portfolios and the Efficient Frontier

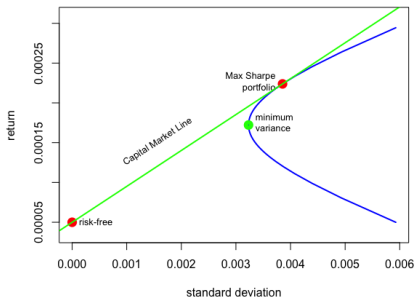
The *maximum Sharpe* portfolios are *efficient portfolios*, so they form the *efficient frontier*.

A *market portfolio* is the portfolio of all the available assets, with weights proportional to their market capitalizations.

The *maximum Sharpe* portfolio is sometimes considered to be the *market portfolio*, because it's the optimal portfolio for the given value of the risk-free rate r_f .

```
> # Calculate the maximum Sharpe portfolios for different risk-free
> detf <- (c11*crr-cr1^2) ## det(fmat)
> raterfv <- retmvm*seq(from=1.3, to=20, by=0.1)
> raterfv <- c(raterfv, retmvm*seq(from=(-20), to=0.7, by=0.1))
> effront <- sapply(raterfv, function(raterf) {
+   ## Calculate the maximum Sharpe mean return
+   reteffm <- (cr1*raterf-crr)/(c11*raterf-cr1)
+   ## Calculate the maximum Sharpe standard deviation
+   stdev <- sqrt((c11*reteffm^2-2*cr1*reteffm+crr)/detf)
+   c(return=reteffm, stdev=stdev)
+ }) ## end sapply
> effront <- effront[, order(effront["return", ])]
> # Plot the efficient frontier
> reteffv <- effront["return", ]
> stdevs <- effront["stdev", ]
> aspectr <- 0.6*max(stdevs)/diff(range(reteffv)) ## Aspect ratio
> plot(x=stdevs, y=reteffv, t="l", col="blue", lwd=2, asp=aspectr,
+   main="Maximum Sharpe Portfolio and Efficient Frontier",
+   xlim=c(0.0, max(stdevs)), xlab="standard deviation", ylab="return")
> # Plot the minimum variance portfolio
> points(x=stdevmv, y=retmvm, col="green", lwd=6)
> text(x=stdevmv, y=retmvm, labels="minimum \nvariance", pos=4, ce=
```

Maximum Sharpe Portfolio and Efficient Frontier



```
> # Calculate the maximum Sharpe return and standard deviation
> raterf <- min(reteffv)
> retmax <- (cr1*raterf-crr)/(c11*raterf-cr1)
> stdevmax <- sqrt((c11*retmax^2-2*cr1*retmax+crr)/detf)
> # Plot the maximum Sharpe portfolio
> points(x=stdevmax, y=retmax, col="red", lwd=6)
> text(x=stdevmax, y=retmax, labels="Max Sharpe\nportfolio", pos=2,
> # Plot the risk-free point
> points(x=0, y=raterf, col="red", lwd=6)
> text(x=0, y=raterf, labels="risk-free", pos=4, cex=0.8)
> # Plot the tangent line
> sharper <- (stdevmax*detf)/(c11*retmax-cr1)
> abline(a=raterf, b=sharper, lwd=2, col="green")
> text(x=0.6*stdevmax, y=0.8*retmax, labels="Capital Market Line",
+   pos=2, cex=0.8, srt=180/pi*atan(aspectr*sharper))
```

Efficient Portfolios and Their Tangent Lines

The *efficient frontier* consists of all the *maximum Sharpe* portfolios corresponding to different values of the risk-free rate.

The target return can be expressed as a function of the risk-free rate as:

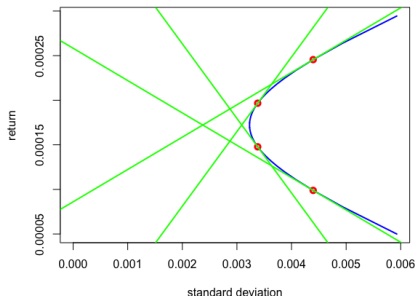
$$r_t = \frac{C_{r1} r_f - C_{rr}}{C_{l1} r_f - C_{r1}}$$

If $r_f \rightarrow \pm\infty$ then $r_t \rightarrow r_{mv} = C_{r1}/C_{l1}$.

But if the risk-free rate tends to the mean returns of the minimum variance portfolio: $r_f \rightarrow r_{mv} = C_{r1}/C_{l1}$, then $r_t \rightarrow \pm\infty$, which means that there is no efficient portfolio corresponding to the risk-free rate equal to the mean returns of the minimum variance portfolio: $r_f = r_{mv} = C_{r1}/C_{l1}$.

```
> # Plot the efficient frontier
> reteffv <- efffront["return", ]
> stdevs <- efffront["stdev", ]
> plot(x=stdevs, y=reteffv, t="l", col="blue", lwd=2,
+      xlim=c(0.0, max(stdevs)),
+      main="Efficient Frontier and Tangent Lines",
+      xlab="standard deviation", ylab="return")
```

Efficient Frontier and Tangent Lines

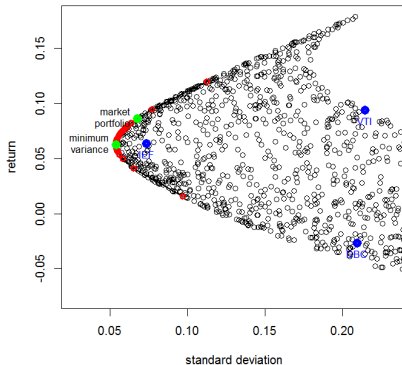


```
> # Calculate vector of mean returns
> reteffv <- min(reteffv) + diff(range(reteffv))*c(0.2, 0.4, 0.6, 0.8)
> # Plot the tangent lines
> for (reteffm in reteffv) {
+   ## Calculate the maximum Sharpe standard deviation
+   stdev <- sqrt((c11*reteffm^2-2*c11*reteffm+crr)/detf)
+   ## Calculate the slope of the tangent line
+   sharper <- (stdev*detf)/(c11*reteffm-c11)
+   ## Calculate the risk-free rate as intercept of the tangent line
+   raterf <- reteffm - sharper*stdev
+   ## Plot the tangent portfolio
+   points(x=stdev, y=reteffm, col="red", lwd=3)
+   ## Plot the tangent line
+   abline(a=raterf, b=sharper, lwd=2, col="green")
+ } ## end for
```

Random Portfolios

```
> # Calculate random portfolios
> nportf <- 1000
> randportf <- sapply(1:nportf, function(it) {
+   weightv <- runif(nstocks-1, min=-0.25, max=1.0)
+   weightv <- c(weightv, 1-sum(weightv))
+   ## Portfolio returns and standard deviation
+   c(return=252*sum(weightv*retm),
+     stdev=sqrt(252*drop(weightv %%% covmat %%% weightv)))
+ }) ## end sapply
> # Plot scatterplot of random portfolios
> x11(widthp <- 6, heightp <- 6)
> plot(x=randportf["stdev", ], y=randportf["return", ],
+   main="Efficient Frontier and Random Portfolios",
+   xlim=c(0.5*stdev, 0.8*max(randportf["stdev", ])),
+   xlab="standard deviation", ylab="return")
> # Plot maximum Sharpe portfolios
> lines(x=effront[, "stdev"], y=effront[, "return"], lwd=2)
> points(x=effront[, "stdev"], y=effront[, "return"],
+   col="red", lwd=3)
> # Plot the minimum variance portfolio
> points(x=stdev, y=retp, col="green", lwd=6)
> text(stdev, retp, labels="minimum\nvariance", pos=2, cex=0.8)
> # Plot efficient portfolio
> points(x=effront[marketp, "stdev"],
+   y=effront[marketp, "return"], col="green", lwd=6)
> text(x=effront[marketp, "stdev"], y=effront[marketp, "return"],
+   labels="market\nportfolio", pos=2, cex=0.8)
```

Efficient Frontier and Random Portfolios



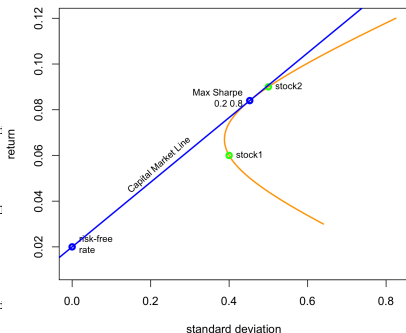
```
> # Plot individual assets
> points(x=sqrt(252*diag(covmat)),
+   y=252*retm, col="blue", lwd=6)
> text(x=sqrt(252*diag(covmat)), y=252*retm,
+   labels=names(retm),
+   col="blue", pos=1, cex=0.8)
```

Efficient Frontier for Two-stock Portfolio

The *efficient frontier* for a two-stock portfolio illustrates its dependence on the correlation and the risk-free rate.

```
> # Define the parameters
> raterf <- 0.02 ## Risk-free rate
> retp <- c(stock1=0.06, stock2=0.09) ## Returns
> stdevs <- c(stock1=0.4, stock2=0.5) ## Standard deviations
> corrp <- 0.6 ## Correlation
> covmat <- matrix(c(1, corrp, corrp, 1), nc=2) ## Covariance matrix
> covmat <- t(t(stdevs*covmat)*stdevs)
> weightv <- seq(from=-1, to=2, length.out=71) ## Weights
> weightv <- cbind(weightv, 1-weightv)
> retport <- weightv %*% retp ## Portfolio returns
> portfsd <- sqrt(rowSums(weightv*(weightv %*% covmat))) ## Portfolio standard deviation
> sharper <- (retport-raterf)/portfsd ## Portfolio Sharpe ratios
> # Plot the efficient frontier
> # x11(widthhp <- 6, heightp <- 5) ## Windows
> dev.new(widthhp <- 6, heightp <- 5, noRStudioGD=TRUE) ## Mac
> plot(portfsd, retport, t="l",
+   main=paste0("Efficient Frontier and CML for Two Stocks\ncorrelat:", corrp),
+   xlab="standard deviation", ylab="return",
+   lwd=2, col="orange", xlim=c(0, max(portfsd)), ylim=c(0.01, max(retport)),
+   # Add the maximum Sharpe portfolio
+   whichmax <- which.max(sharper)
+   sharperem <- max(sharper) ## Maximum Sharpe ratio
+   retmax <- retport[whichmax]
+   sdeff <- portfsd[whichmax]
+   weightm <- round(weightv[whichmax], 2)
+   points(sdeff, retmax, col="blue", lwd=3)
+   text(x=sdeff, y=retmax, labels=paste(c("Max Sharpe\n",
+     structure(c(weightm, (1-weightm)), names=c("stock1", "stock2")),
+     pos=2, cex=0.8))
```

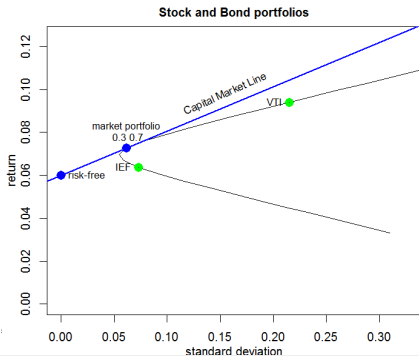
Efficient Frontier and CML for Two Stocks
correlation = 60%



```
> # Plot individual stocks
> points(stdevs, retp, col="green", lwd=3)
> text(stdevs, retp, labels=names(retp), pos=4, cex=0.8)
> # Add point at risk-free rate and draw Capital Market Line
> points(x=0, y=raterf, col="blue", lwd=3)
> text(0, raterf, labels="risk-free\nrate", pos=4, cex=0.8)
> abline(a=raterf, b=sharperem, lwd=2, col="blue")
> rangev <- par("usr")
> text(sdeff/2, (retmax+raterf)/2,
+   labels="Capital Market Line", cex=0.8, , pos=3,
+   srt=45*atan(sharperem*(rangev[2]-rangev[1])/
+     (rangev[4]-rangev[3])*heightp/widthp)/(0.25*pi))
```

Efficient Frontier of Stock and Bond Portfolios

```
> # Vector of symbol names
> symbolv <- c("VTI", "IEF")
> # Matrix of portfolio weights
> weightv <- seq(from=-1, to=2, length.out=31)
> weightv <- cbind(weightv, 1-weightv)
> # Calculate portfolio returns and volatilities
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> retport <- retp %>% t(weightv)
> portfv <- cbind(252*colMeans(retport),
+ sqrt(252)*matrixStats::colSDs(retport))
> colnames(portfv) <- c("returns", "stdev")
> raterf <- 0.06
> portfv <- cbind(portfv,
+ (portfv[, "returns"]-raterf)/portfv[, "stdev"])
> colnames(portfv)[3] <- "Sharpe"
> whichmax <- which.max(portfv[, "Sharpe"])
> sharpem <- portfv[whichmax, "Sharpe"]
> plot(x=portfv[, "stdev"], y=portfv[, "returns"],
+ main="Stock and Bond portfolios", t="l",
+ xlim=c(0, 0.7*max(portfv[, "stdev"])), ylim=c(0, max(portfv[,
+ "returns"])), xlab="standard deviation", ylab="return")
> # Add blue point for efficient portfolio
> points(x=portfv[whichmax, "stdev"], y=portfv[whichmax, "returns"])
> text(x=portfv[whichmax, "stdev"], y=portfv[whichmax, "returns"],
+ labels=paste(c("efficient portfolio\n",
+ structure(c(weightv[whichmax, 1], weightv[whichmax, 2]), names=
+ pos=3, cex=0.8)
```



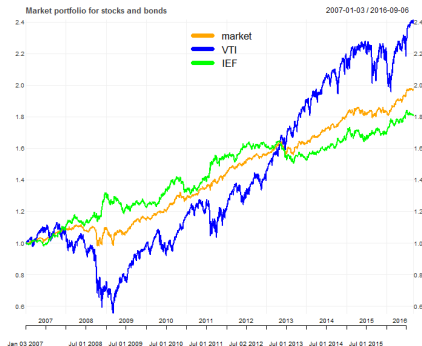
```
> # Plot individual stocks
> retm <- 252*sapply(retport, mean)
> stdevs <- sqrt(252)*sapply(retport, sd)
> points(stdevs, retm, col="green", lwd=6)
> text(stdevs, retm, labels=names(retport), pos=2, cex=0.8)
> # Add point at risk-free rate and draw Capital Market Line
> points(x=0, y=raterf, col="blue", lwd=6)
> text(0, raterf, labels="risk-free", pos=4, cex=0.8)
> abline(a=raterf, b=sharpem, col="blue", lwd=2)
> rangev <- par("usr")
> text(max(portfv[, "stdev"])/3, 0.75*max(portfv[, "returns"]),
+ labels="Capital Market Line", cex=0.8, , pos=3,
+ srt=45*atan(sharpem*(rangev[2]-rangev[1])/
+ (rangev[4]-rangev[3])*
+ heightp/widthp)/(0.25*pi))
```

Performance of Efficient Portfolio for Stocks and Bonds

```

> # Calculate cumulative returns of VTI and IEF
> retsoptim <- lapply(retp, function(retp) exp(cumsum(retp)))
> retsoptim <- rutils::do_call(cbind, retsoptim)
> # Calculate the efficient portfolio returns
> retsoptim <- cbind(exp(cumsum(retp %*%
+   c(weightv[whichmax], 1-weightv[whichmax]))),
+   retsoptim)
> colnames(retsoptim)[1] <- "efficient"
> # Plot efficient portfolio with custom line colors
> themev <- chart_theme()
> themev$col$line.col <- c("orange", "blue", "green")
> chart_Series(retsoptim, theme=themev,
+   name="Efficient Portfolio for Stocks and Bonds")
> legend("top", legend=colnames(retsoptim),
+   cex=0.8, inset=0.1, bg="white", lty=1,
+   lwd=6, col=themev$col$line.col, bty="n")

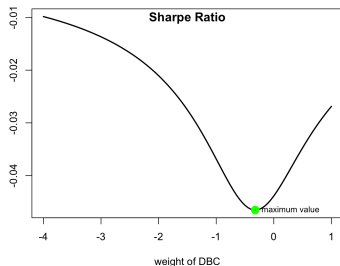
```



Sharpe Ratio Objective Function

The function `optimize()` performs *one-dimensional* optimization over a single independent variable, in the specified interval.

```
> # Calculate daily ETF percentage returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> nrows <- NROW(retp)
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Objective equal to minus Sharpe ratio
> objfun <- function(weightv, retp) {
+   retportf <- retp %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else
+     return(-mean(retportf)/stdev)
+ } ## end objfun
> # Objective for equal weight portfolio
> objfun(weightv, retp=retp)
> optiml <- unlist(optimize(f=function(weightv)
+   objfun(c(1, 1, weightv), retp=retp),
+   interval=c(-10, 10)))
> # Vectorize objective function with respect to third weight
> objvec <- function(weightv) sapply(weightv,
+   function(weightv) objfun(c(1, 1, weightv), retp=retp))
> # Or
> objvec <- Vectorize(FUN=function(weightv)
+   objfun(c(1, 1, weightv), retp=retp),
+   vectorize.args="weightv") ## end Vectorize
> objvec(1)
> objvec(1:3)
```



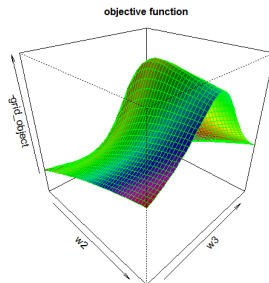
```
> # Plot objective function with respect to third weight
> curve(expr=objvec, type="l", xlim=c(-4.0, 1.0),
+   xlab=paste("weight of", names(weightv[3])),
+   ylab="", lwd=2)
> title(main="Sharpe Ratio", line=(-1)) ## Add title
> points(x=optiml[1], y=optiml[2], col="green", lwd=6)
> text(x=optiml[1], y=optiml[2],
+   labels="maximum value", pos=4, cex=0.8)
>
> ### Below is simplified plotting of objective function
> # Create vector of DBC weights
> weightv <- seq(from=-4, to=1, by=0.1)
> objv <- sapply(weightv, function(weightv)
+   objfun(c(1, 1, weightv), retp))
> plot(x=weightv, y=objv, t="l",
+   xlab="weight of DBC", ylab="", lwd=2)
> title(main="Sharpe Ratio", line=(-1)) ## Add title
> points(x=optiml[1], y=optiml[2], col="green", lwd=6)
> text(x=optiml[1], y=optiml[2],
```

Perspective Plot of Portfolio Objective Function

The function `persp()` plots a 3d perspective surface plot of a function specified over a grid of argument values.

The function `outer()` calculates the values of a function over a grid spanned by two variables, and returns a matrix of function values.

The package *rgl* allows creating *interactive* 3d scatterplots and surface plots including perspective plots, based on the *OpenGL* framework.



```
> # Vectorize function with respect to two weights
> objvec <- Vectorize(
+   FUN=function(w1, w2, w3) objfun(c(w1, w2, w3), retp),
+   vectorize.args=c("w2", "w3")) ## end Vectorize
> # Calculate objective on 2-d (w2 x w3) parameter grid
> w2 <- seq(-3, 7, length=50)
> w3 <- seq(-5, 5, length=50)
> gridm <- outer(w2, w3, FUN=objvec, w1=1)
> rownames(gridm) <- round(w2, 2)
> colnames(gridm) <- round(w3, 2)
> # Perspective plot of objective function
> persp(w2, w3, -gridm,
+   theta=45, phi=30, shade=0.5,
+   col="rainbow(50)", border="green",
+   main="objective function")
```

```
> # Interactive perspective plot of objective function
> library(rgl)
> rgl::persp3d(z=-gridm, zlab="objective",
+   col="green", main="objective function")
> rgl::persp3d(
+   x=function(w2, w3) {objvec(w1=1, w2, w3)},
+   xlim=c(-3, 7), ylim=c(-5, 5),
+   col="green", axes=FALSE)
> # Render the 3d surface plot of function
> rgl::rglwidget(elementId="plot3drgl", width=1000, height=1000)
```

Portfolio Optimization Using optim()

The functional `optim()` performs *multi-dimensional* optimization.

The argument `par` are the initial parameter values.

The argument `fn` is the objective function to be minimized.

The argument of the objective function which is to be optimized, must be a vector argument.

`optim()` accepts additional parameters bound to the dots `"..."` argument, and passes them to the `fn` objective function.

The arguments `lower` and `upper` specify the search range for the variables of the objective function `fn`.

`method="L-BFGS-B"` specifies the quasi-Newton optimization method.

The parameter `control=list(factr=1e5)` determines the precision of the L-BFGS-B method. A smaller `factr` value produces results with a higher precision, but it requires more iterations and a longer optimization. A larger `factr` value achieves a faster solution but less precise results.

`optim()` returns a list containing the location of the minimum and the objective function value.

```
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Optimization to find weights with maximum Sharpe ratio
> optiml <- optim(par=weightv,
+               fn=objfun,
+               retp=retp,
+               method="L-BFGS-B",
+               control=list(factr=1e5),
+               upper=c(10, 10, 10),
+               lower=c(-10, -10, -10))
> # Optimal parameters
> weightv <- optiml$par
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
> # Optimal Sharpe ratio
> -objfun(weightv, retp)
> # Calculate the weights from the inverse covariance matrix
> weightv <- drop(solve(cov(retp)) %*% sapply(retp, mean))
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Optimization Algorithms Using `optim()`

Table: Comparison of optimization methods for portfolio optimization

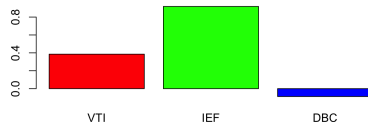
Method	Type	Strengths	Weaknesses	Suitability for Portfolio Optimization
BFGS	Quasi-Newton	Fast convergence, uses gradient + Hessian approximation	Memory-heavy for large problems	Strong choice for medium-sized portfolios
L-BFGS-B	Limited-memory BFGS with box constraints	Handles bounds (e.g., weights ≥ 0), memory-efficient	Requires smooth objective	Best overall for portfolio optimization with constraints
CG (Conjugate Gradient)	Gradient-based	Efficient for large-scale problems	Slower convergence than BFGS	Good for very large portfolios
Nelder-Mead	Derivative-free simplex	Works without gradients, robust for small problems	Slow, scales poorly in high dimensions	Limited use; not ideal for large portfolios
Brent	1D derivative-free	Very efficient in one dimension	Only works for single-variable problems	Not applicable to portfolios
SANN (Simulated Annealing)	Stochastic global search	Escapes local minima, works on non-smooth functions	Very slow, no guarantee of optimality	Useful for highly non-convex problems

Optimized Portfolio Returns

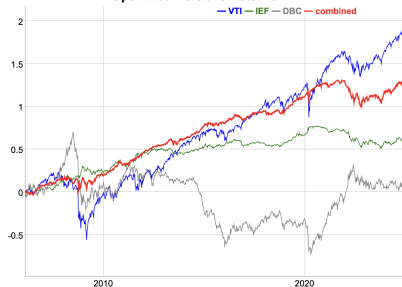
The optimized portfolio has both long and short positions, and has a larger *Sharpe ratio* than the individual assets.

```
> # barplot of optimal portfolio weights
> barplot(weightv, col=c("red", "green", "blue"),
+   main="Optimized portfolio weights")
> # Calculate the cumulative wealth of the optimized portfolio
> wealthv <- cbind(retp %*% weightv, retp)
> colnames(wealthv)[1] <- "combined"
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   (mean(x)-raterf)/c(Sharpe=sd(x), Sortino=sd(x[x<0]))))
> # Plot the log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> colv <- colnames(wealthv)
> colr <- c("red", "blue", "green", "grey")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Optimized Portfolio Returns") %>%
+   dyOptions(colors=colr, strokeWidth=1) %>%
+   dySeries(name=colv[1], col="red", strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Optimized portfolio weights



Optimized Portfolio Returns



Package *DEoptim* for Global Optimization

The function `DEoptim()` from package *DEoptim* performs *global* optimization using the *Differential Evolution* algorithm.

Differential Evolution is a genetic algorithm which evolves a population of solutions over several generations,

<https://link.springer.com/content/pdf/10.1023/A:1008202821328.pdf>

The first generation of solutions is selected randomly.

Each new generation is obtained by combining solutions from the previous generation.

The best solutions are selected for creating the next generation.

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

Gradient optimization methods are more efficient than *Differential Evolution* for smooth objective functions with no local minima.

```
> # Rastrigin function with vector argument for optimization
> rastrigin <- function(vecv, param=25){
+   sum(vecv^2 - param*cos(vecv))
+ } ## end rastrigin
> vecv <- c(pi/6, pi/6)
> rastrigin(vecv=vecv)
> library(DEoptim)
> # Optimize rastrigin using DEoptim
> optim1 <- DEoptim::DEoptim(rastrigin,
+   upper=c(6, 6), lower=c(-6, -6),
+   DEoptim.control(trace=FALSE, itermax=50))
> # Optimal parameters and value
> optim1$optim$bestmem
> rastrigin(optim1$optim$bestmem)
> summary(optim1)
> plot(optim1)
```

Portfolio Optimization Using Package *DEoptim*

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

With the parameter `control=list(parallelType=1)`, `DEoptim::DEoptim()` performs the optimization using parallel computing.

With the parameter `control=list(itermax=100)`, `DEoptim::DEoptim()` performs at most 100 iterations.

To achieve a good precision, `itermax` should be set to a much larger value, such as 10000. 100 iterations.

```
> # Perform optimization using DEoptim
> optim1 <- DEoptim::DEoptim(fn=objfun,
+   upper=rep(10, NCOL(retp)),
+   lower=rep(-10, NCOL(retp)),
+   retp=retp,
+   control=list(trace=FALSE, itermax=100, parallelType=1))
> weightv <- optim1$optim$bestmem
> names(weightv) <- colnames(retp)
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Efficient Portfolios With Ridge Regularization

Portfolio optimization has a very large number of parameters, equal to the number of stocks in the portfolio. This causes the *overfitting* of the stock portfolio weights to the in-sample returns.

The technique of *regularization* is designed to reduce the overfitting of the parameters in a model.

Ridge regularization adds a penalty term to the objective function, proportional to the sum of squares of the weights: $\mathbf{w}^T \mathbf{w} = \sum_{i=1}^n w_i^2$.

The Lagrangian for the efficient portfolio weights with *ridge regularization* is:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C} \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} + 1) - \lambda_2 (\mathbf{w}^T \mathbf{r} - r_t) + \lambda \mathbf{w}^T \mathbf{w}$$

Where the parameter λ determines the amount of regularization.

The Lagrangian can be rewritten using the shrinkage covariance matrix $\mathbb{C}_r = \mathbb{C} + \lambda \mathbf{1}$:

$$\mathcal{L} = \mathbf{w}^T \mathbb{C}_r \mathbf{w} - \lambda_1 (\mathbf{w}^T \mathbf{1} - 1) - \lambda_2 (\mathbf{w}^T \mathbf{r} - r_t)$$

This is the efficient portfolio Lagrangian with the shrinkage covariance matrix $\mathbb{C}_r$.

Maximum Sharpe Portfolio With Ridge Regularization

Maximizing the efficient portfolio Lagrangian is equivalent to maximizing the *Sharpe ratio*:

$$SR = \frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma}$$

Where $\boldsymbol{\mu} = \mathbf{r} - r_f$ is the vector of excess returns, and $\sigma = \sqrt{\mathbf{w}^T \mathbb{C}_r \mathbf{w}}$ is the standard deviation of portfolio with the shrinkage covariance matrix $\mathbb{C}_r$.

The portfolio weights satisfying the linear constraint $\mathbf{w}^T \mathbf{1} = 1$ are:

$$\mathbf{w} = \frac{\mathbb{C}_r^{-1} \boldsymbol{\mu}}{\mathbf{1}^T \mathbb{C}_r^{-1} \boldsymbol{\mu}}$$

These are the weights of the *maximum Sharpe* portfolio, with the vector of mean excess returns equal to $\boldsymbol{\mu}$, and the covariance matrix equal to $\mathbb{C}_r$.

```
> # Objective equal to minus ridge Sharpe ratio
> objfun <- function(weightv, retx, lambdaf=0) {
+   retportf <- retx %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else
+     return(-mean(retportf)/stdev + lambdaf*sum(weightv^2))
+ } ## end objfun
> # Create initial vector of portfolio weights
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> # Calculate vector of excess returns
> raterf <- 0.03/252
> retx <- (retp - raterf)
> # Calculate the weights using optimization
> lambdaf <- 300.0
> objfun(weightv, retp=retx, lambdaf=lambdaf)
> optim1 <- optim(par=weightv,
+   fn=objfun,
+   retx=retx,
+   lambdaf=lambdaf,
+   method="L-BFGS-B",
+   upper=c(10, 10, 10),
+   lower=c(-10, -10, -10))
> weightv <- optim1$par
> names(weightv) <- symbolv
> # Calculate ridge covariance matrix of returns and its inverse
> covmat <- cov(retp) + lambdaf*diag(ncol(retp))
> covinv <- solve(a=covmat)
> # Calculate the maximum Sharpe weights
> retm <- colMeans(retx)
> weightms <- drop(covinv %*% retm)/sum(covinv %*% retm)
> all.equal(weightv, weightms)
```

Portfolio Optimization With *Elastic Net Regularization*

The technique of *shrinkage (regularization)* is designed to reduce the number of parameters in a model, for example in portfolio optimization.

The *shrinkage* technique adds a penalty term to the objective function.

The penalty term in *ridge* regularization is the sum of squares of the weights: $\sum_{i=1}^n w_i^2$. The *ridge* penalty shrinks (reduces) the weights together, without shrinking some of them to zero.

The penalty term in *LASSO* regularization is the sum of the absolute values of the weights: $\sum_{i=1}^n |w_i|$. The *LASSO* penalty shrinks the less important weights to zero, similar to dimension reduction.

The *elastic net* regularization is a combination of *ridge* regularization and *LASSO* regularization:

$$w_{\max} = \arg \max_w \left[\frac{\mathbf{w}^T \boldsymbol{\mu}}{\sigma} - \lambda \left((1 - \alpha) \sum_{i=1}^n w_i^2 + \alpha \sum_{i=1}^n |w_i| \right) \right]$$

The λ parameter determines the amount of shrinkage, and α controls the balance between the *ridge* and *LASSO* penalties.

```
> # Objective with shrinkage penalty
> objfun <- function(weightv, retp, lambdaf, alphaf) {
+   retportf <- retp %*% weightv
+   stdev <- sd(retportf)
+   if (stdev == 0)
+     return(0)
+   else {
+     penaltyv <- lambdaf*((1-alphaf)*sum(weightv^2) +
+     alphaf*sum(abs(weightv)))
+     return(-mean(retportf)/stdev + penaltyv)
+   } ## end if
+ } ## end objfun
> # Objective for equal weight portfolio
> weightv <- rep(1, NROW(symbolv))
> names(weightv) <- symbolv
> lambdaf <- 0.5 ; alphaf <- 0.5
> objfun(weightv, retp=retp, lambdaf=lambdaf, alphaf=alphaf)
> # Perform optimization using DEoptim
> optim1 <- DEoptim::DEoptim(fn=objfun,
+   upper=rep(10, NCOL(retp)),
+   lower=rep(-10, NCOL(retp)),
+   retp=retp,
+   lambdaf=lambdaf,
+   alphaf=alphaf,
+   control=list(trace=FALSE, itermax=100, parallelType=1))
> weightv <- optim1$optim$bestmem
> names(weightv) <- colnames(retp)
> weightv <- weightv/sqrt(sum(weightv^2))
> weightv
```

Optimal Stock Portfolio Weights

The portfolio weights obtained from inverting the covariance matrix are the same as the weights obtained from portfolio optimization.

```
> # Load stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RData")
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] ## Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> # Select 100 random stocks
> retp <- retp[, sample(NCOL(retp), 100)]
> symbolv <- colnames(retp)
> datev <- zoo::index(retp)
> retis <- retp["/2014"] ## In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) ## Excess returns
> # Calculate the maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retx, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
```

```
> # Calculate the weights using optimization without shrinkage
> lambdaf <- 0.0 ; alphaf <- 1.0
> optim1 <- optim(par=weightv,
+               fn=objfun,
+               retp=retp,
+               lambdaf=lambdaf,
+               alphaf=alphaf,
+               method="L-BFGS-B",
+               control=list(factr=1e5),
+               upper=c(10, 10, 10),
+               lower=c(-10, -10, -10))
> weighto <- optim1$par
> names(weighto) <- symbolv
> all.equal(weighto, weightv)
```

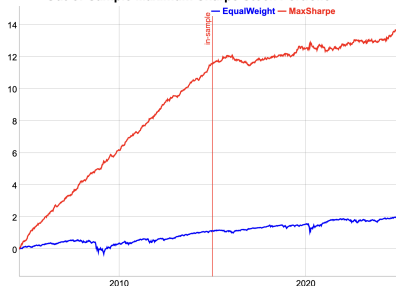
Optimal Stock Portfolio Out-of-Sample

The *portfolio optimization* strategy for stocks is *overfit* in the *in-sample* interval.

Therefore the strategy performance is poor in the *out-of-sample* interval.

```
> # Load S&P 500 stock returns
> load("/Users/jerzy/Develop/lecture_slides/data/sp500_returns.RData")
> datev <- zoo::index(na.omit(retstock$GOOGL))
> retp <- retstock[datev] ## Subset the returns to GOOGL
> # Remove the stocks with any NA values
> numna <- sapply(retp, function(x) sum(is.na(x)))
> retp <- retp[, numna==0]
> # Select 100 random stocks
> retp <- retp[, sample(NCOL(retp), 100)]
> symbolv <- colnames(retp)
> datev <- zoo::index(retp)
> retis <- retp["/2014/"] ## In-sample returns
> raterf <- 0.03/252
> retx <- (retis - raterf) ## Excess returns
> # Maximum Sharpe weights in-sample interval
> colmeanv <- colMeans(retx, na.rm=TRUE)
> covmat <- cov(retx, use="pairwise.complete.obs")
> invreg <- MASS::ginv(covmat)
> weightv <- drop(invreg %*% colmeanv)
> names(weightv) <- symbolv
> # Calculate the maximum Sharpe portfolio returns
> pnlmaxs <- HighFreq::mult_mat(weightv, retp)
> pnlmaxs <- rowMeans(pnlmaxs, na.rm=TRUE)
> pnlmaxs <- xts::xts(pnlmaxs, datev)
> retew <- xts::xts(rowMeans(retp, na.rm=TRUE), datev)
> pnlmaxs <- pnlmaxs*sd(retew["/2014/"])/sd(pnlmaxs["/2014/"])
> # Calculate the shrinkage portfolio returns
> lambdaf <- 1.0 ; alphaf <- 1.0
> optiml <- optim(par=weightv,
+               fn=objfun,
+               retp=retx,
```

Out-of-Sample Maximum Sharpe Stock Portfolio



```
> wealthv <- cbind(pnlmaxs, pnls)
> colnames(wealthv) <- c("MaxSharpe", "Shrink")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2014/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv["/2015/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of cumulative portfolio returns
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Out-of-Sample Maximum Sharpe Stock Portfolio") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(end(retis[1]), label="in-sample", strokePattern="solid")
+   dyLegend(width=300)
```

Optimal Portfolios Under Zero Correlation

If the correlations of returns are equal to zero, then the covariance matrix is diagonal:

$$\mathbb{C} = \begin{pmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^2 \end{pmatrix}$$

Where σ_i^2 is the variance of returns of asset i .

The inverse of $\mathbb{C}$ is then simply:

$$\mathbb{C}^{-1} = \begin{pmatrix} \sigma_1^{-2} & 0 & \cdots & 0 \\ 0 & \sigma_2^{-2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_n^{-2} \end{pmatrix}$$

The *minimum variance* portfolio weights are proportional to the inverse of the individual variances:

$$w_i = \frac{1}{\sigma_i^2 \sum_{i=1}^n \sigma_i^{-2}}$$

The *maximum Sharpe* portfolio weights are proportional to the ratio of the excess returns divided by the individual variances:

$$w_i = \frac{\mu_i}{\sigma_i^2 \sum_{i=1}^n \mu_i \sigma_i^{-2}}$$

The portfolio weights are proportional to the *Kelly ratios* - the excess returns divided by the variances:

$$w_i \propto \frac{\mu_i}{\sigma_i^2}$$

Maximum Return Portfolio Using Linear Programming

The stock weights of the maximum return portfolio are obtained by maximizing the portfolio returns:

$$w_{max} = \arg \max_w [\bar{\mathbf{r}}^T \mathbf{w}] = \arg \max_w \left[\sum_{i=1}^n w_i r_i \right]$$

Where $\mathbf{r}$ is the vector of returns, and $\mathbf{w}$ is the vector of portfolio weights, with a linear constraint:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

And a box constraint:

$$0 \leq w_i \leq 1$$

The weights of the maximum return portfolio can be calculated using linear programming (LP), which is the optimization of linear objective functions subject to linear constraints.

The function `Rglpk_solve_LP()` from package *Rglpk* solves linear programming problems by calling the *GNU Linear Programming Kit* library.

```
> library(rutils)
> library(Rglpk)
> # Vector of symbol names
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> # Calculate the objective vector - the mean returns
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> objvec <- colMeans(retp)
> # Specify matrix of linear constraint coefficients
> coeffm <- matrix(c(rep(1, nstocks), 1, 1, 0),
+                 nc=nstocks, byrow=TRUE)
> # Specify the logical constraint operators
> logop <- c("=", "<=")
> # Specify the vector of constraints
> consv <- c(1, 0)
> # Specify box constraints (-1, 1) (default is c(0, Inf))
> boxc <- list(lower=list(ind=1:nstocks, val=rep(-1, nstocks)),
+             upper=list(ind=1:nstocks, val=rep(1, nstocks)))
> # Perform optimization
> optiml <- Rglpk::Rglpk_solve_LP(
+   obj=objvec,
+   mat=coeffm,
+   dir=logop,
+   rhs=consv,
+   bounds=boxc,
+   max=TRUE)
> all.equal(optiml$optimum, sum(objvec*optiml$solution))
> optiml$solution
> coeffm %*% optiml$solution
```

Conditional Value at Risk (CVaR)

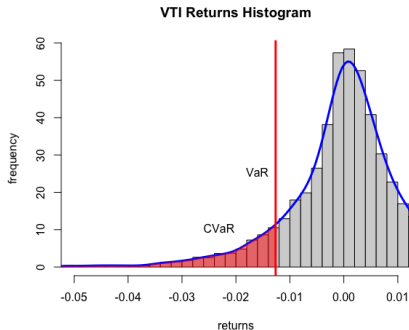
The *Conditional Value at Risk (CVaR)* is equal to the average of the *VaR* for confidence levels less than a given confidence level α :

$$\text{CVaR} = \frac{1}{\alpha} \int_0^\alpha \text{VaR}(p) dp$$

The *Conditional Value at Risk* is also called the *Expected Shortfall (ES)*, or the *Expected Tail Loss (ETL)*.

The function `density()` calculates a kernel estimate of the probability density for a sample of data, and returns a list with a vector of loss values and a vector of corresponding densities.

```
> # Calculate the VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> confl <- 0.1
> varisk <- quantile(retp, confl)
> cvar <- mean(retp[retp < varisk])
> # Or
> sortv <- sort(as.numeric(retp))
> varind <- round(confl*NROW(retp))
> varisk <- sortv[varind]
> cvar <- mean(sortv[1:varind])
> # Plot histogram of VTI returns
> varmin <- (-0.05)
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=100, xlim=c(varmin, 0.01),
+   ylab="frequency", freq=FALSE, main="VTI Returns Histogram")
```



```
> # Plot density of losses
> densv <- density(retp, adjust=1.5)
> lines(densv, lwd=3, col="blue")
> # Add line for VaR
> abline(v=varisk, col="red", lwd=3)
> ymax <- max(densv$y)
> text(x=varisk, y=2*ymax/3, labels="VaR", lwd=2, pos=2)
> # Add shading for CVaR
> rangev <- (densv$x < varisk) & (densv$x > varmin)
> polygon(
+   c(varmin, densv$x[rangev], varisk),
+   c(0, densv$y[rangev], 0),
+   col=rgb(1, 0, 0, 0.5), border=NA)
> text(x=1.5*varisk, y=ymax/7, labels="CVaR", lwd=2, pos=2)
```

CVaR Portfolio Weights Using Linear Programming

The stock weights of the minimum CVaR portfolio can be calculated using linear programming (LP), which is the optimization of linear objective functions subject to linear constraints,

$$w_{min} = \arg \max_{\mathbf{w}} \left[\sum_{i=1}^n w_i b_i \right]$$

Where b_i is the negative objective vector, and $\mathbf{w}$ is the vector of portfolio weights, with a linear constraint:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

And a box constraint:

$$0 \leq w_i \leq 1$$

The function `Rglpk_solve_LP()` from package *Rglpk* solves linear programming problems by calling the *GNU Linear Programming Kit* library.

```
> library(rutils) ## Load rutils
> library(Rglpk)
> # Vector of symbol names and returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- colMeans(rutils::etfenv$returns[, symbolv])
> retm <- colMeans(retp)
> confl <- 0.05
> rmin <- 0 ; wmin <- 0 ; wmax <- 1
> weightsum <- 1
> ncols <- NCOL(retp) ## number of assets
> nrows <- NROW(retp) ## number of rows
> # Create objective vector
> objvec <- c(numeric(ncols), rep(-1/(confl/nrows), nrows), -1)
> # Specify matrix of linear constraint coefficients
> coeffm <- rbind(cbind(rbind(1, retm),
+                        matrix(data=0, nrow=2, ncol=(nrows+1))),
+                 cbind(coredata(retp), diag(nrows), 1))
> # Specify the logical constraint operators
> logop <- c("=", ">=", rep(">=", nrows))
> # Specify the vector of constraints
> consv <- c(weightsum, rmin, rep(0, nrows))
> # Specify box constraints (wmin, wmax) (default is c(0, Inf))
> boxc <- list(lower=list(ind=1:ncols, val=rep(wmin, ncols)),
+              upper=list(ind=1:ncols, val=rep(wmax, ncols)))
> # Perform optimization
> optiml <- Rglpk_solve_LP(obj=objvec, mat=coeffm, dir=logop, rhs=consv,
+                          box=boxc, all.equal=TRUE, sum=objvec*optiml$solution)
> all.equal(optiml$optimum, sum(objvec*optiml$solution))
> coeffm %*% optiml$solution
> as.numeric(optiml$solution[1:ncols])
```

Package *quadprog* for Quadratic Programming

Quadratic programming (*QP*) is the optimization of quadratic objective functions subject to linear constraints.

Let $O(x)$ be an objective function that is quadratic with respect to a vector variable x :

$$O(x) = \frac{1}{2}x^T Qx - d^T x$$

Where Q is a *positive definite* matrix ($x^T Qx > 0$), and d is a vector.

An example of a *positive definite* matrix is the covariance matrix of linearly independent variables.

Let the linear constraints on the variable x be specified as:

$$Ax \geq b$$

Where A is a matrix, and b is a vector.

The function `solve.QP()` from package *quadprog* performs optimization of quadratic objective functions subject to linear constraints.

```
> library(quadprog)
> # Minimum variance weights without constraints
> optim1 <- solve.QP(Dmat=2*covmat,
+                   dvec=rep(0, 2),
+                   Amat=matrix(0, nr=2, nc=1),
+                   bvec=0)
> # Minimum variance weights sum equal to 1
> optim1 <- solve.QP(Dmat=2*covmat,
+                   dvec=rep(0, 2),
+                   Amat=matrix(1, nr=2, nc=1),
+                   bvec=1)
> # Optimal value of objective function
> t(optim1$solution) %*% covmat %*% optim1$solution
> ## Perform simple optimization for reference
> # Objective function for simple optimization
> objfun <- function(x) {
+   x <- c(x, 1-x)
+   t(x) %*% covmat %*% x
+ } ## end objfun
> unlist(optimize(f=objfun, interval=c(-1, 2)))
```

Portfolio Optimization Using Package *quadprog*

The objective function is designed to minimize portfolio variance and maximize its returns:

$$O(x) = \mathbf{w}^T \mathbb{C} \mathbf{w} - \mathbf{w}^T \mathbf{r}$$

Where $\mathbb{C}$ is the covariance matrix of returns, $\mathbf{r}$ is the vector of returns, and $\mathbf{w}$ is the vector of portfolio weights.

The portfolio weights $\mathbf{w}$ are constrained as:

$$\mathbf{w}^T \mathbf{1} = \sum_{i=1}^n w_i = 1$$

$$0 \leq w_i \leq 1$$

The function `solve.QP()` has the arguments:

`Dmat` and `dvec` are the matrix and vector defining the quadratic objective function.

`Amat` and `bvec` are the matrix and vector defining the constraints.

`meq` specifies the number of equality constraints (the first `meq` constraints are equalities, and the rest are inequalities).

```
> # Calculate daily ETF percentage returns
> symbolv <- c("VTI", "IEF", "DBC")
> nstocks <- NROW(symbolv)
> retp <- na.omit(rutils::etfenv$returns[, symbolv])
> # Calculate the covariance matrix
> covmat <- cov(retp)
> # Minimum variance weights, with sum equal to 1
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=numeric(3),
+                               Amat=matrix(1, nr=3, nc=1),
+                               bvec=1)
> # Minimum variance, maximum returns
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=apply(0.1*retp, 2, mean),
+                               Amat=matrix(1, nr=3, nc=1),
+                               bvec=1)
> # Minimum variance positive weights, sum equal to 1
> a_mat <- cbind(matrix(1, nr=3, nc=1),
+                 diag(3), -diag(3))
> b_vec <- c(1, rep(0, 3), rep(-1, 3))
> optim1 <- quadprog::solve.QP(Dmat=2*covmat,
+                               dvec=numeric(3),
+                               Amat=a_mat,
+                               bvec=b_vec,
+                               meq=1)
```

Homework Assignment

Required

Study all the lecture slides in `FRE7241_Lecture_5.pdf`, and run all the code in `FRE7241_Lecture_5.R`

Recommended

- Read about *optimization methods*:
Bolker Optimization Methods.pdf
Yollin Optimization.pdf
Boudt DEoptim Large Portfolio Optimization.pdf
- Read about *PCA* in:
pca-handout.pdf
pcaTutorial.pdf